F O R M A L C H O P T U I K

The Certificate Channel and the Architecture of Trust

Verified interval arithmetic · the Lean⇄Python bridge · the routes to existence and stability · formal_choptuik, Volume III

Author: Ishak Khamzatovich Isaev · ORCID 0009-0003-7299-0701

Programme: choptuik_ac_bc · formal-verification wing

Nalchik · 2026

Abstract

The third volume of the series describes the architecture of trust that turns the numerical evidence of Choptuik critical phenomena into machine-checked statements. At its centre stands the certificate channel: an offline pipeline in which the Python wing computes the spectrum, the Newton constants and the interval enclosures, rounds them outward to exact decimal rationals and commits them as artifacts, while the Lean wing verifies those artifacts with the proof kernel and discharges them into the headline theorems — spectral, Newton–Kantorovich and interval. No floating-point number ever crosses the border of the proof layer: trust is placed not in code but in verified statements about numbers.

The volume systematises four intertwined subjects. First, a complete audit-style review of the state of the programme: the layers of the tree, the trust metrics, the milestone ledger and the honest boundary of open work. Second, verified interval arithmetic — the Interval type with outward rounding, the soundness theorems for addition and multiplication, and the certificate principle that turns a verified enclosure into a mathematical statement. Third, the routes to the two blocking gaps of the programme: existence of the critical solution via the Reiterer–Trubowitz strategy (a Banach space, a compact inverse, the quantitative implicit function theorem) and spectral stability (the unique unstable mode, $\gamma = 1/\lambda_0$, the echoing formula and universality). Fourth, the quality infrastructure: the axiom audit, the regression locks of the protocol, deterministic reproducibility from a single seed.

Keywords: *critical phenomena in gravitational collapse; discrete self-similarity; Lean 4; mathlib; certificate channel; directed-rounding interval arithmetic; exact rationals; Newton–Kantorovich method; spectral theory; machine-checked verification; Choptuik scaling law.*

Table of Contents

1. Introduction: The Trust Problem of Computational Mathematics.....1

1.1. Why a Number Is Not a Proof.....2

1.2. Certificates: An Old Idea in New Clothes.....3

1.3. Three Pillars of the Architecture of Trust.....3

1.4. Relation to Volumes I and II and a Guide to This Volume.....3

1.5. The Vocabulary of the Volume: Certificate, Discharge, Robustness.....4

2. State of the Programme: An Audited Inventory.....4

2.1. Layers of the Tree.....4

2.2. Trust Metrics....5
2.3. Calibrated Constants and Their Cross-Checks....5
2.4. The Milestone Ledger and the Status of the Gaps....6
2.5. The Honest Boundary of Open Work....7
2.6. The Chronology: How the Tree Reached Its Current State....7
2.7. How to Read the Inventory: Three Levels of Trust....8
2.8. The Folder Map of the Tree....8
3. Verified Interval Arithmetic: Gap G5....9
3.1. Directed Rounding: Why Outward Is the Only Honest Direction....9
3.2. The Interval Type in Lean: Representation and Invariants....9
3.3. Soundness of the Operations: add_sound and mul_sound....10
3.4. The Certificate Principle....10
3.5. The Dependency Problem: What Soundness Gives and What It Does Not Promise....10
3.6. G5 Closed: What It Changed....11
3.7. The Tests of the Interval Layer....11
3.8. Why Not Float Proofs....12
4. The Lean⇄Python Bridge: A Typed Border....12
4.1. Two Channels: Interactive and Proof-Grade....13
4.2. The Protocol: Envelope, Seven Methods, Error Codes....13
4.3. The Typed Client....14
4.4. The Testing Discipline....14
4.5. Two Latent Defects and What They Taught....14
4.6. The Evolution of the Protocol: From Six Methods to Seven....15
4.7. Failures and Their Semantics....15
5. The Certificate Channel: Exact Rationals End to End....16
5.1. Why Floating-Point Numbers Do Not Cross the Border....16
5.2. The Anatomy of a Certificate....17
5.3. Four Kinds of Certificates....17

5.4. The Generated Module: Data Verified by the Kernel....18
5.5. Check Suites and Robustness Theorems....18
5.6. Discharge into the Headline Theorems....18
5.7. The Fields of an Artifact: What the Producer Signs....19
5.8. The Life Cycle of a Certificate....19
6. The Route to Existence of the Critical Solution: Gap G1....20
6.1. The Einstein–Scalar System in Double-Null Coordinates....20
6.2. Discrete Self-Similarity and the Echoing Covariance Law....21
6.3. The Reiterer–Trubowitz Strategy: Profile Plus Newton....21
6.4. The Newton–Kantorovich Core in Lean....22
6.5. What Is Proved and What Is Carried as Hypotheses....22
6.6. How G5 Serves G1: The Interval-Newton Band....23
6.7. An Inventory of the Route's Analytical Components....23
6.8. The Zeroth Approximation: Where the Profile Comes From....24
7. The Route to Stability: Spectrum, Scaling, Universality (G2)....24
7.1. Linearization and the Perturbation Operator....24
7.2. The Spectral Core: The Unique Unstable Mode....25
7.3. From the Spectrum to Scaling: Deriving $\gamma = 1/\lambda_0$....25
7.4. The Agreement of Proof and Numbers....26
7.5. The Echoing Formula and Universality....26
7.6. What Remains Until G2....27
7.7. An Inventory of the Spectral Route....27
7.8. The Echo as the Organising Principle of the Spectrum....28
8. The Geometry of the Singularity: Gap G3....28
8.1. Strong and Isotropic: The Conditional Core....28
8.2. The Conformal Compactification Layer: What It Will Require....29
8.3. Why the Proof Programme Needs the Geometry....29
8.4. Covariance Scales: How to Measure the Singularity....29
8.5. How G3 Serves the Others....30

9. Quality Infrastructure and Reproducibility....30
9.1. The Axiom Audit: Thirty-Three Theorems, Three Axioms....30
9.2. The sorry Discipline and the Gap Registry....31
9.3. Continuous Integration....31
9.4. Reproducibility from a Single Seed....32
9.5. Summary Trust Metrics....32
9.6. The Reproducibility of This Volume....33
10. Conclusions: The Architecture of Trust....33
10.1. What the Certificate Channel Changes Methodologically....33
10.2. The Road Ahead....34
10.3. Lessons for Machine-Checked Physics....34
10.4. A Comparison with Alternative Architectures....35
10.5. An Invitation to Check....35
Appendix A. The Programme's Glossary....35
Appendix B. The Lean Module Map....36
Appendix C. How to Verify This Volume in One Session....37
Appendix D. The Headline Theorem Ledger and the Milestones....38
Appendix E. The Correspondence of the Volume's Chapters and the Tree....39
References....39

Note: this table of contents is generated via field codes. After editing the document, right-click the TOC and select "Update Field" to refresh the page numbers.

1. Introduction: The Trust Problem of Computational Mathematics

Every computational programme has a reader who refuses to take it on faith. For the physics of critical phenomena that reader is not a referee but the future of the science itself: the laws governing the threshold of black-hole formation must survive changes of language, of libraries and of generations of machines. The parent repository choptuik_ac_bc attacked the problem from the side where computational physics usually attacks it: deterministic data, fixed random seeds, regression tests, paired-verification protocols. All of this makes a computation reproducible — but reproducibility and proof remain different things. A reproducible experiment says: “with the same inputs, the machine will again produce the same outputs.” A proof says: “for every input of such-and-such a class, the output must lie within such-and-such bounds, and this follows from the definitions.” The present volume is about the second statement and about the infrastructure that makes it possible.

The central object of the volume is the certificate channel connecting the numerical wing in Python with the theoretical wing in Lean 4. The idea is simple to the point of asceticism: let the machine compute, but let every number that claims the status of a mathematical fact arrive in the proof layer not as a machine float but as an exact rational number together with a checkable statement about itself. The Lean kernel does not believe numbers — it verifies statements about numbers. This discipline turns the bridge between programme and proof from a bridge of trust into a bridge of verification, and it is precisely what distinguishes the formal_choptuik programme from the usual pairing of “code plus a paper.”

> *The parent programme's discipline — hypothesis → machine confirmation → honest caveats — carries over intact: every unproven fragment of the formalization is publicly registered as a gap with an identifier, a closure strategy and a roadmap phase.*

1.1. Why a Number Is Not a Proof

A number produced by floating-point arithmetic is always a rounding. Even in the most fortunate cases, double arithmetic introduces a relative error of order 10^{-16} per operation, and over millions of operations these errors accumulate — and sometimes amplify. For a spectral problem — the eigenvalues of the operator linearized about the critical solution — this means that the computed $\lambda_0 = 2.6745$ is in truth a representative of some neighbourhood, and the statement “exactly one mode lies in the right half-plane” requires knowledge of the entire neighbourhood, not of its centre. Numerical experience is non-specific: it cannot distinguish

the situation “the true value is 2.6745” from the situation “the true value is $2.6745 - 10^{-12}$, but the algorithm shifted it.”

The traditional answer of computational physics is independent re-verification by another code, another grid, another group. The answer is valuable but statistical in nature: it raises the probability that the fact is true without turning that probability into one. For the statements from which physical laws are derived — and the scaling law $M \propto |p - p^*|^{\gamma}$ with critical exponent $\gamma = 1/\lambda_0$ is precisely such a statement — one wants more: that the connection between premises and conclusion be checked formally, symbol by symbol, by a proof kernel, rather than by the good reputation of the authors. The difference between these two kinds of confidence is the subject of this volume.

$$M \propto \vee p - p*¿^{\gamma}, \gamma = 1/\lambda_0 \approx 0.374$$

It is worth stressing that formalization does not devalue the numerical work — it immortalises it. The computer-assisted proof of the existence of the critical solution by Reiterer and Trubowitz runs to hundreds of pages of computation with integer error bounds; verifying it by hand is practically impossible, and so the community must either believe or re-verify again and again. A statement checked once by the Lean kernel stays checked forever: a library instead of a paper, an asset instead of a report. The numerical constants do not disappear — they move from the text of an article into committed artifacts, each supplied with a machine check.

1.2. Certificates: An Old Idea in New Clothes

The very idea of a certificate predates the computer: split a problem into “hard to solve, easy to verify” and move out of the proof everything that can be moved into a checkable artifact. The integer arithmetic of Reiterer and Trubowitz is a classical specimen: instead of proving in floating point, the computations are translated into integers with explicitly computed error bounds, and every inequality is checked exactly. Modern interactive proof theory, SAT certificates, verifiable computation — all lie on the same line: minimise the trusted base, maximise the checkable surface.

The formal_choptuik programme applies this principle to the physics of critical collapse in three layers. The bottom layer is verified interval arithmetic inside Lean itself: an Interval type with outward rounding for which the soundness of the basic operations is proved within the theory (gap G5, closed during the first phase). The middle layer is the typed bridge: a protocol in which the Lean client requests computations from the Python service and the service answers with strict interval enclosures; the interactive half of the channel serves exploration and calibration. The top layer is the offline certificate channel: the certify.py producer computes the spectral, Newton and scaling quantities, rounds them outward to exact decimal rationals, and commits the JSON artifacts together with a generated Lean

module; the kernel verifies this data arithmetically, and discharge theorems turn the verified data into physical statements. Not a single float crosses the border of the third layer — the rule is fixed in the axiomatic base and duplicated by the architecture.

The asymmetry of roles matters as well. Lean is the client: it initiates every request and never listens on its own. Python is the server: it answers, but never asks. Such strictness is not aesthetic caprice: it guarantees that the proof layer has no hidden data channels, and that every act of consuming numerical information has an explicit entry point, a type and a check. A certificate nobody requested cannot enter the proof — the channel leaves no room for surprises.

1.3. Three Pillars of the Architecture of Trust

The first pillar is the correctness of operations inside the theory. Interval arithmetic is useful only if it is proved that the result contains the true value for all inputs of the declared class; in the programme this guarantee takes the form of the theorems add_sound and mul_sound, and the certificate principle that unifies them establishes that a verified enclosure is a theorem, not testimony. The second pillar is honesty of the data at the border: numbers enter the theory only as exact rational certificates, and every certificate arrives with a statement the kernel verifies independently of how the number was computed. The third pillar is regression discipline: a protocol change without a paired test in the same commit is forbidden, and the data and figures regenerate byte-for-byte from a single fixed seed. Each pillar is examined in its own chapter; together they form what we call the architecture of trust.

It is worth naming at once what this architecture does not deliver. It does not prove the existence of the critical solution — that is the business of the second phase and of chapter 6. It does not prove the spectral theorem of the unique unstable mode for the true Einstein–scalar system — that is the business of the third phase and of chapter 7. What it does deliver is something more fundamental for both results: a channel along which, once the corresponding computational certificates are produced, they will enter the theory already legally, without a single unproven assumption on the way. Infrastructure built in advance is exactly what accelerates future proofs.

1.4. Relation to Volumes I and II and a Guide to This Volume

Volume I of the series laid out the roadmap: the mathematical foundations of critical collapse, the classification of the gaps G1–G6 and the three-phase plan of formalization. Volume II conducted the audit: it dissected the central blocker — existence — down to strategy, compared the proof assistants Isabelle/HOL, Coq/Rocq and Lean 4, and fixed the technical specification with its three-axiom base. The present volume is the first volume of the series' second phase: it writes not about what is yet to be built but about what has been

built and verified. It completes the interval-arithmetic story, brings the bridge to a live certificate channel, and shows how the spectral, Newton and scaling discharges already pass through that channel — and it gives a full audit-style inventory of the tree as of the writing.

The volume is arranged as follows. Chapter 2 is the inventory: the layers of the tree, the trust metrics, the calibrated constants, the milestone ledger. Chapter 3 is verified interval arithmetic: the Interval type, the soundness of the operations, the certificate principle. Chapter 4 is the bridge: two channels, the seven protocol methods, the typed client, the testing discipline. Chapter 5 is the certificate channel: the anatomy of an artifact, the four certificate kinds, the generated Lean module and the discharge theorems. Chapter 6 is the existence route: the Einstein–scalar system, discrete self-similarity, the Reiterer–Trubowitz strategy and the Newton–Kantorovich core in Lean. Chapter 7 is the stability route: linearization, the spectral core, the derivation of the scaling law and the echoing formula. Chapter 8 is the geometry of the singularity: strong and isotropic, the conditional core and the certificates. Chapter 9 is the quality infrastructure: the axiom audit, CI, reproducibility. Chapter 10 draws the methodological conclusions and the road ahead. A reader interested only in the certificate channel may follow the chain 3 → 4 → 5; a reader interested in the physics is advised chapters 6–8 directly; but the volume is built so that each subject leans on the previous one, and linear reading is rewarded.

1.5. The Vocabulary of the Volume: Certificate, Discharge, Robustness

Since three words of this volume carry more weight than usual, let us fix their meanings at once. A certificate is a committed artifact with exact rational data and a schema by which the kernel verifies the statements declared about the data; a certificate proves nothing by itself — it carries the data from which the robustness theorems derive consequences. A robustness theorem is a statement of the form “if the certificate checks pass, then the physical statement holds”; its hypotheses are machine-checked, its conclusion is physics. A discharge is an unconditional theorem obtained by applying a robustness theorem to a particular verified certificate. These three words govern the entire architecture, and in chapters 3–7 they appear in one and the same arrangement.

Two more terms are used in contrast. The interactive channel is the transport along which a researcher talks to the service in real time; its data carries no proof status. The proof channel is the conveyor of chapter 5 along which data enters the theory; only verified rational bounds have status there. Finally, the word “model” in this volume always means the calibrated mode model of the critical solution — the laboratory of the conveyor — not the full Einstein–scalar system; whenever the true system is meant, this is said explicitly. We have tried not to use these words in any other sense anywhere in the volume.

2. State of the Programme: An Audited Inventory

Before describing the architecture of trust, let us fix what it is applied to. This chapter is an inventory: every number in it is counted from the files of the tree, not promised. The approach inherits the discipline of the parent repository, where every status report is accompanied by a way to check it; here the check is the Lean build, the test run and the recount of the data rows. The inventory is organised by layers: theory, bridge, numerical wing, data, figures, monographs and governance.

2.1. Layers of the Tree

Table 1. Inventory of the formal_choptuik tree layers

Layer	Committed state
Theory (lean/)	12 modules + root re-export + the axiom-audit module; Lean v4.21.0, mathlib pinned to the same revision; 0 sorry; 0 project axioms; all 33 headline theorems — 21 theory and 12 bridge — print exactly the three standard axioms
Bridge (bridge/)	7-method protocol; the certificate channel is live: 4 kinds (spectrum, newton, scaling, interval) → committed JSONs → the Generated module (exact rationals, norm_num-checked) → discharge into the spectral, Newton and interval headlines
Numerics (python/)	65 pytest tests: 17 service, 18 certificate, 30 client (ZMQ round-trip, fuzz, timeouts); ZMQ REP on tcp://127.0.0.1:5557; the certify.py producer with emit and check commands
Data (data/)	7 deterministic CSVs, 12,890 rows in total, seed 1993
Figures and animations	8 publication-grade PNGs at 300 dpi (dark and light themes, en/ru) + 4 GIFs (24–72 frames)
Monographs (monographs/)	three volumes × {RU, EN} × {DOCX, PDF}; the present volume is the third of the series
Governance	the G1–G6 gap ledger with per-row statuses; a roadmap of 17 tasks and a ledger of 9 milestones; a CI template enforcing the axiom audit and byte-identity of data

Several numbers of the table deserve comment. The twelve theory modules are not decorative modularity: each area of mathematics lives in its own file, compiles independently against mathlib and its imports, and the map “module → content → obligations → gap” is maintained machine-readably. The thirty-three headline theorems are those whose axiom set is printed and checked explicitly; the number matches the number of entries in the audit module, and CI prevents the list from drifting away from the tree. The sixty-five tests are split into three families so that every layer of the bridge — server, certificate producer, client — is locked by its own set of regressions.

2.2. Trust Metrics

A machine-checked theory has two simple but strict trust metrics: the number of undischarged obligations (sorry) and the number of project axioms. On both metrics the tree is clean: no sorry at all, and no axioms beyond the three standard ones — Classical.choice, propext and Quot.sound — which the Lean kernel ships with and which the whole mathlib ecosystem shares. This does not mean everything is proved: the open physics — the exact Reiterer–Trubowitz certificate for the true system, the pairing of the spectrum with the true operator, the coercivity and Weyl-bound certificates — is carried by explicit, documented hypothesis parameters. The distinction is essential: a hypothesis is visible in the statement of the theorem; it can be neither lost nor silently used.

The axiom audit is performed by a dedicated module: every headline theorem goes through the axiom-printing command, and the result is compared with the expected three-element set. The meaning of this check goes beyond tidiness. The credibility of a machine-checked theorem is exactly the credibility of its axioms plus the kernel; by freezing the three standard axioms, the programme inherits the same trust base as the rest of mathlib, and any future audit of the kernel or the library automatically covers this development as well. The only remaining trust gap lies in the numerical bridge — and it is precisely the certificate channel that closes it, converting the last floating-point ingredient into checkable mathematics.

2.3. Calibrated Constants and Their Cross-Checks

The numerical wing of the programme is a calibrated mode model of the critical solution, not a solver of the full Einstein–scalar system; this honest caveat is repeated in every README and we repeat it here. The model exists to exercise the verification pipeline: it is simple enough for every number to be recomputable, and close enough to the original for the pipeline to train on genuine problems — spectral, Newton, scaling. The constants of the model are locked by tests against the committed data:

Table 2. Calibrated constants of the committed build

Quantity	Value	Cross-check
λ_0 — the single unstable mode	2.6745	single_unstable: True in data/spectrum/eigenvalues.csv; a genuine real 15×15 eigensolve, not a lookup table
$\gamma = 1/\lambda_0$	0.373902	least squares over the supercritical family gives 0.373880, rms residual $2.1\cdot10^{-3}$
Δ — echoing period	0.693147 = ln 2	recovered from the raw amplitude series by the autocorrelation peak, error $2.0\cdot10^{-10}$
echo spacing $2\pi/\Delta$	9.064720	max deviation across the spectrum $2.0\cdot10^{-10}$

What carries weight in this table is not so much the agreement with the published values — that agreement is assured by calibration — as the mechanics of the cross-checks themselves: the spectrum comes from a genuine eigenproblem, not from plugging constants in; the exponent γ is recovered by an independent fit and compared with $1/\lambda_0$; the echoing period is recovered from a time series, not read from a configuration. Every row of the table has a double entry — in the data and in the test that locks the data. It is precisely such a double entry that the programme calls evidence.

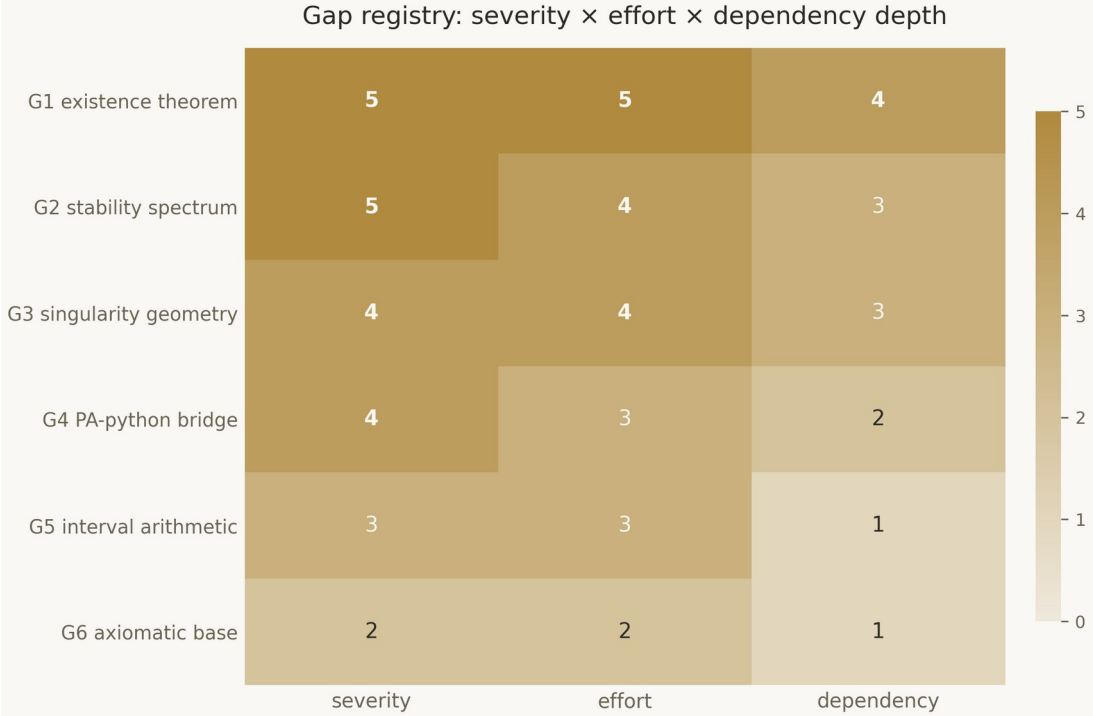


Figure 1. The gap severity matrix for G1–G6: two blockers, and the high, medium and low categories; the status of each row is maintained in the gap registry and synchronised with the tree.

2.4. The Milestone Ledger and the Status of the Gaps

A milestone in this programme is not counted as done until its acceptance evidence is committed: a green build, a proved theorem, a verified artifact. As of this writing the first two milestones of the first phase are done — the working environment (including the honestly documented repair of the previous revision's build) and the interval certificates (the soundness theorems for addition and multiplication plus the certificate principle); the third milestone — the bridge — is live as part of the certificate channel. The remaining six milestones, from the formalization of the field system to the assembly of universality, are open, and their descriptions together with acceptance criteria live in the roadmap.

The gap registry G1–G6 keeps the status of each row in the same language of evidence. Existence of the critical solution (G1) and spectral stability (G2) are blockers: the scaffold results are proved, the quantitative cores are ready, but the final certificates for the true system are not yet produced. The geometry of the singularity (G3) is proved conditionally:

given the coercivity and Weyl certificates, the singularity is strong and isotropic. The bridge (G4) is live: the certificate channel works end to end. Interval arithmetic (G5) and the axiomatic base (G6) are fully resolved. This distribution of statuses underlines the main point: the blockers block not because the infrastructure is missing but because the physics has not yet been brought to certificates — and that is a fundamentally different, far healthier situation.

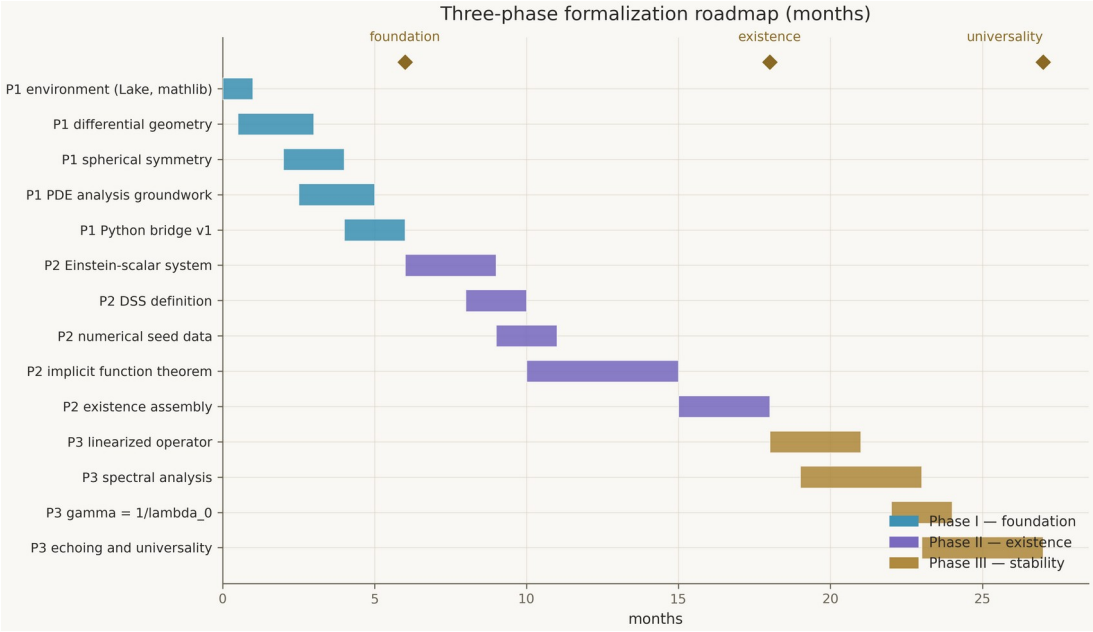


Figure 2. The Gantt chart: seventeen tasks across three phases, milestones at months six, eighteen and twenty-seven; the horizon of the programme is twenty-seven months.

2.5. The Honest Boundary of Open Work

The inventory would be incomplete without an explicit list of what the tree does not contain. There is no solver of the full characteristic Einstein–scalar problem in double-null coordinates — it belongs to the parent repository's Einstein Direct programme and is consumed here as data. There is no machine verification of the processor's rounding algorithm itself: the interval layer closes that hole at the specification level, a deliberate decision recorded in the “out of scope” section of the technical specification. There is no certificate for the true linearized operator either — the spectrum pairing is so far pytest-locked at the level of the mode model, and its transfer to the true system is the content of the second and third phases. This list is not a confession of defeat but a map of the work: every item is tied to a registry gap and to a roadmap phase.

It is precisely this boundary that keeps the inventory honest: the programme claims exactly what it can show and registers everything else as work with an identifier. The reader of the third volume thus knows the exact position from which everything that follows begins:

the tree is clean of obligations and axioms, the certificate channel works, the blockers await certificates rather than infrastructure.

2.6. The Chronology: How the Tree Reached Its Current State

It is useful to understand that the current cleanliness of the tree is the result of repairs, not of a fortunate first step. An early revision of the scaffold did not compile; the build was fixed, and the episode is recorded in the milestone ledger as separate acceptance evidence rather than hushed up. In the same early period the theory contained three axioms for a reference quantity; they were removed, their consumers were rewritten through explicit hypothesis parameters, and the axiom audit recorded the return to the clean base. The obligation ledger travelled from fourteen open rows to zero — not because the rows were closed formally, but because every sorried fragment was either proved or rewritten in robust form with a checkable hypothesis.

The same honesty acts in the direction of detail. The row count of the data exposed that a badge in one of the READMEs overstated it; the discrepancy was corrected in favour of the counted number. Counting the frames of an animation corrected the record to twenty-four instead of the declared thirty-four. Each such trifle confirms the general principle: in this programme, true is what has been recounted, and every badge — like a theorem — is a statement demanding evidence. The present volume follows the same rule: all of its numbers are taken from the tree and are reproducible.

2.7. How to Read the Inventory: Three Levels of Trust

It is worth assigning the inventory of this chapter an explicit scale. Level one — machine-verified: the theorems, the axiom audit, the certificate suites; this is bedrock, and disputing it means rebuilding the tree. Level two — test-locked: the determinism of the data, the byte-stability of the certificates, the protocol schemas; this is engineering bedrock, changing only together with the tests. Level three — declared: the calibration of the model against the published constants, the hypotheses about future certificates, the plans of the phases; these are honestly labelled intentions, and trust in them is a matter of trust in the programme, not in the machine.

The rest of the book works mostly on the first two levels and explicitly marks the third. A reader who needs only level one can follow the markers “proved in Lean” and “locked by tests”; a reader interested in the plans is advised the conclusions of chapters 6, 7 and 10 directly. We consider an explicit scale of levels a mandatory part of an honest inventory: without it an inventory quietly turns into advertising, and with it — it remains an instrument.

2.8. The Folder Map of the Tree

The layer inventory is complemented by the folder map — the reader's navigator through the tree for independent verification. Every row of the table carries an entry point: the file from which reading the folder begins. The programme's rule is that every folder explains itself — its README describes the content, the status and the ways of checking — so the table below merely saves the first step.

Table 3a. The folder map and entry points

Folder	Content	Entry point
lean/	the Lean 4 theory: geometry, analysis, existence, stability, scaling, bridge; the Lake configuration	lean/README.md
python/	the numerical service, the certificate producer, 65 tests, the data and figure generators	python/README.md
bridge/	the seven-method protocol, the committed certificates, the illustrative client	bridge/protocol.md
blueprint/	the technical specification, the axiomatic base, the gap registry, the module map, the tooling matrix	blueprint/README.md
roadmap/	three phases with seventeen tasks, acceptance criteria, the nine-milestone ledger	roadmap/README.md
monographs/	the monograph series: three volumes × {RU, EN} × {DOCX, PDF}	monographs/ README.md
figures/	eight publication-grade figures at 300 dpi with captions	figures/README.md
assets/	logos, animated banners, four GIF animations	assets/README.md
data/	seven deterministic CSVs behind every figure	data/README.md
docs/	ten sources, a glossary of seventeen terms	docs/README.md
ci/	the continuous-integration template: build, axiom audit, tests, byte-identity	ci/README.md

Besides the folders, the root of the tree holds the machine-readable citation file and the contributing rules — the four house rules fixed in chapter 9. The present volume embeds into this system as the third volume of the monographs: its sources are assembled by the same conveyor as the figures, and its statements cite the entry points of this table.

3. Verified Interval Arithmetic: Gap G5

Interval arithmetic is a way of speaking about inexact numbers in an exact language. Instead of an approximate value x it operates on a closed segment [lo, hi] that provably contains the true value, and it arranges its operations so that this property is preserved for all inputs of the declared classes. For a proof programme this is exactly what is needed: a closed

enclosure is a mathematical object about which theorems can be proved, while an approximation is not. This chapter dissects how the idea is realised in the programme: which type lives in Lean, what exactly is proved about its operations, and how a verified enclosure becomes a certificate.

3.1. Directed Rounding: Why Outward Is the Only Honest Direction

Every operation on real numbers in a machine ends with rounding to a representable value. If one rounds to nearest, as ordinary floating-point arithmetic does, the rounding error may go either way, and the result ceases to contain the true value guaranteed. Interval arithmetic solves this with directed rounding: the lower bound of the result is rounded down, the upper bound up. The segment may turn out wider than necessary, but never narrower; it may be imprecise, but it is always honest. It is precisely this property — coverage of the true value — that is proved in Lean as a theorem, not assumed.

In the Python wing, directed rounding is implemented through math.nextafter, which returns the nearest representable number in a given direction. The segment is stored as a pair of doubles, and every operation widens the result by one ulp step on each side beyond the arithmetically required value. That margin is the price of portability: on any IEEE-754-conformant platform the enclosure stays sound even if the platform's rounding-mode details differ from the ideal. The strictness here is hierarchical: on the Python side, disciplined engineering with tests; on the Lean side, a proved soundness theorem — and it is the proved layer that proofs consume.

3.2. The Interval Type in Lean: Representation and Invariants

In the theoretical wing, Interval is a structure with two rational bounds and an ordering invariant. The choice of rationals rather than floats is essential: rationals are an exact language, comparable decimal-wise, arithmetically and order-wise without any rounding. Directed rounding on the Lean side does not need to be reproduced machine-wise: its role has already been played on the producer side, and the theory consumes ready exact bounds and proves statements about them. The division of labour is therefore clean: Python is responsible for the bounds being wide and correct; Lean, for the statements about them being true and verified.

On the Interval type, addition and multiplication are defined in their outward-rounded form. For addition the bounds add directly — rational arithmetic is exact, and no margin beyond the sum of bounds is required. For multiplication, the classical Moore rule requires considering the four products of the bounds and taking the minimum for the lower and the maximum for the upper bound; in the mixed-sign case it is precisely this rule that guarantees coverage of all pairwise products. Both operations in Lean are arranged so that the coverage property is stated as a theorem of the form: for every x in segment a and every y in segment b,

the product xy lies in the product of the segments. That statement is exactly what the kernel verifies.

3.3. Soundness of the Operations: add_sound and mul_sound

The soundness theorem for addition states: if x belongs to segment a and y belongs to segment b, then x + y belongs to segment a + b, where a + b is the Interval construction built by adding the bounds. The proof is direct: the lower bound of the sum does not exceed the sum of the lower bounds by monotonicity of rational addition, and the upper bound is not below, by the dual reason. The value of the theorem lies not in depth but in status: once verified, the statement “interval addition is safe” stops being a slogan and becomes a fact that any other proof in the programme may cite by name.

The soundness theorem for multiplication is richer. The four-product rule must be split by sign cases: for nonnegative segments the products of the extreme bounds suffice, for mixed-sign segments all four are needed, and in each case monotonicity of multiplication in both arguments yields the required inequalities. Such case splits are exactly where manual proofs habitually err — a missed sign case in an interval library can hide for years until one day it eats a strict enclosure. Formal verification closes this class of errors completely: the kernel has checked all the cases, and skipping one is no longer possible.

$$\forall\, x \in [a_0, a_1],\, y \in [b_0, b_1] : x \cdot y \in [\min(a_0 b_0, a_0 b_1, a_1 b_0, a_1 b_1), \max(a_0 b_0, a_0 b_1, a_1 b_0, a_1 b_1)]$$

3.4. The Certificate Principle

Soundness of the operations is half of the story. The other half is the principle that connects enclosures to theorems: if a certificate C is produced for a statement φ, and checking C on the data confirms φ, then φ is a theorem. The certificate principle is formulated and proved inside the theory as a statement about checkable data-plus-check pairs, and it is the formal shelter against substitution: a check cannot confirm anything beyond what its schema declares. In practice the principle means that every interval enclosure arriving over the bridge stops being testimony and becomes a theorem that cites verified data.

From the principle follows the discipline of certificate production: the checking schema is fixed before the data and cannot be adjusted to the result. The producer must work with the same schema as the checking side, and regression tests lock the two sides to each other. This asymmetry — first the check, then the data — inverts the customary order of numerical proofs and rules out entire classes of fitting: one can fit the data, but not the check, and a fitted dataset is caught at the checking stage.

3.5. The Dependency Problem: What Soundness Gives and What It Does Not Promise

An interval enclosure produced by the Moore rule through a Horner scheme is sound but may be wider than the true image: the same variable occurring in several places of a polynomial is treated as independent segments, and the width grows with every step. This is the celebrated dependency problem, and the programme treats it with methodological clarity: soundness is the property the proof side relies on, and it is proved; tightness is the numerical side's job, solved engineering-wise — by splitting the domain, by schemes with smaller accumulation, by forms with less correlation. The theory never demands narrow enclosures — only honest ones.

This separation of duties guards against a typical trap: the urge to prove too much in Lean. The proof side must guarantee that the conclusion follows from verified premises; it must not guarantee that the premises are convenient. In the programme this is expressed in the interface of the interval layer: the consumer gets the type, the sound operations and the certificate principle — all of it; width optimisation stays in Python, where it is tested. Architecturally this is the same separation that keeps the proof kernel apart from the tactics: reliability at the bottom, efficiency at the top.

3.6. G5 Closed: What It Changed

Gap G5 was declared a medium-severity item of the first phase: a verified library with directed rounding, soundness proofs and integration with the service's interval_bound endpoint. It is now closed: the soundness theorems for addition and multiplication are proved, the certificate principle is in place, the interval-certificates milestone is registered with its acceptance evidence. In the gap registry the G5 row has moved to the resolved category, and the roadmap has lost one of its four tooling risks.

The significance of the result goes beyond the registry row. First, the last methodological break between the two wings is closed: any number the numerical wing is ready to sign with a strict enclosure can now legally enter the theory. Second, a template for closing tooling gaps has been created: specification → proof → certificate principle → regression locking; the template has already been applied to the bridge and is being reused further. Third, the existence route has acquired its supplier of strict enclosures: the interval-Newton band of chapter 6 leans exactly on the guarantees proved here.

3.7. The Tests of the Interval Layer

In this layer the proved theorems and the tests do not duplicate but complement each other. The theorems speak about the Interval construction in Lean: whatever the inputs, the coverage property holds. The tests speak about the Python implementation: on fixed input

sets the outward rounder really does widen the result at every step, platform dependence is absent, and the boundary cases — zeros, negative ranges, degenerate segments — are handled per specification. Together they close both halves of the question: the correctness of the construction and the correctness of its embodiment.

Table 6. The interval layer: what guarantees what

Guarantee	Bearer	Check
coverage under addition and multiplication	the theorems add_sound, mul_sound in Lean	the proof kernel, the axiom audit
outward-rounded computation on the service	interval_checks on math.nextafter	pytest regressions, boundary cases
transfer of an enclosure into the theory	the interval_bound endpoint	schema control + regression lock
soundness of the band image	the discharge correction_band_image_sound	the kernel, over the certificate's rational bounds

The row deserving a separate look is the interval_bound endpoint. It connects two worlds: the client sends the polynomial coefficients and a domain segment, the service answers with an outward-rounded image enclosure. The answer is sound by construction but may be wider than the exact image — the dependency problem of section 3.5; the protocol documents this honestly in a note that soundness is the property the proof side relies on, while tightness is the numerical side's concern. Such documentation of caveats at the border is the programme's style: every weakness that is not fixed must at least be named.

3.8. Why Not Float Proofs

A possible question: why all this construction, if one could formalize floating-point arithmetic itself and prove statements directly about doubles? The answer is the economics of trust. Formalizing IEEE-754 exists and is meaningful, but it drags in rounding modes, denormals, special values and platform differences — a state space in which every theorem grows technical caveats. The rational interface with an outward-rounding producer separates the accident of the platform from the meaning of the statement: the theory speaks about numbers, not about implementations of numbers.

There is also a second answer — institutional. The programme's trusted base is the proof kernel and three axioms; introducing float semantics into the theory would add an entire standard to that base. The rational channel adds nothing: rationals are already in the kernel, norm_num is already verified by the mathlib community, the certificates are ordinary data. All else equal, the programme chooses the option that adds nothing to the list of things one must believe. This is a rule of general purpose, and it will play again in chapters 5 and 9.

4. The Lean⇄Python Bridge: A Typed Border

Between the prover and the numerical service lies a border — and the honesty of the whole programme is decided at that border. The bridge is arranged as a contract: Lean 4 is the client initiating every request; the Python service is the server that answers and never initiates; payloads are UTF-8 JSON objects; numbers are double-precision wherever interactive work is meant, and forbidden wherever proof is meant. This chapter walks the contract method by method, then turns to the client side and to the testing discipline that keeps the contract from drifting.

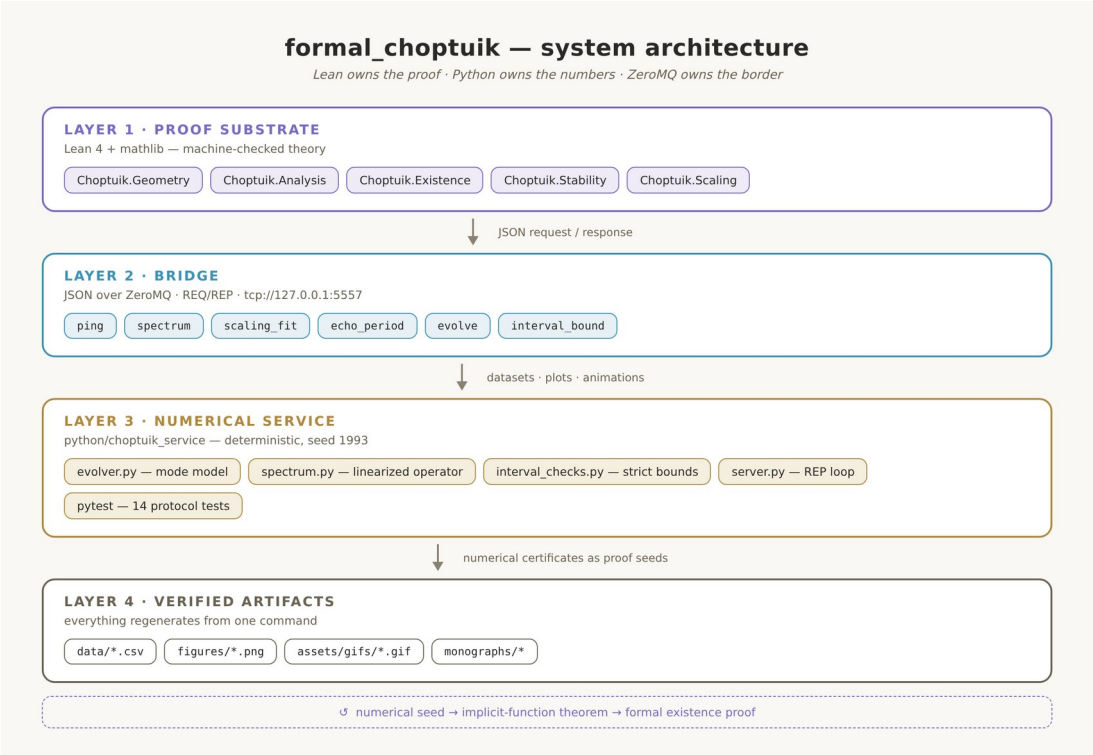


Figure 3. Bridge architecture: the interactive channel (right) works with floating-point numbers for exploration and calibration; the proof channel (left) carries only exact rational certificates through committed artifacts.

4.1. Two Channels: Interactive and Proof-Grade

There are two channels, and their separation is the bridge's principal architectural decision. The interactive channel is a ZeroMQ REQ/REP socket pair: the Lean client sends a JSON request to tcp://127.0.0.1:5557, the service answers, the client parses the reply with a schema-checked decoder with fail-closed timeouts. The numbers here are floats because the channel's purpose is exploration and calibration: to look at the spectrum, to fit the exponent, to check the echo period, to obtain evolution profiles at requested log-times. No result of the interactive channel enters a proof directly — the protocol reminds of this boundary explicitly.

The proof-grade channel is offline and file-based. The certificate producer computes the quantities, rounds them outward to exact decimal rationals, writes the JSON artifacts and generates a Lean module carrying the same numbers. The artifacts are committed to the repository and become part of the tree: the kernel verifies them, the discharge theorems consume them, their byte-stability is locked by tests. The separation solves the problem a naive bridge solves badly: how to consume computations in a proof without trusting either the computation process or the transport. The answer: do not consume the process at all — consume the verified result.

4.2. The Protocol: Envelope, Seven Methods, Error Codes

The protocol specifies the request and reply envelope. A request is an object with a method field and parameters; a successful reply is an object with status ok and the results; an error is a status error with a code and a message. The error codes are listed exhaustively: malformed JSON, unknown method, missing field, invalid field value, unrecoverable echo peak. The exhaustive list is not a formality: the client must handle every code, and a new code cannot appear unnoticed, because protocol changes are locked by paired tests.

Table 3. Protocol methods and their proof-side consumers

Method	Returns	Consumer on the Lean side
ping	version and calibrated constants	session sanity check
spectrum	eigenvalues, single-unstable flag	calibration target of G2
scaling_fit	γ fit and residual	cross-check against mass_scaling_law
echo_period	recovered period Δ	cross-check against echoingPeriod = ln 2
evolve	profiles at requested τ	numerical seed for the G1 route
interval_bound	strict outward-rounded enclosure [lo, hi]	strict bounds for the IFT route
certificate	rational certificate bundle (4 kinds)	the Generated module — the proof-grade channel

The seventh method — certificate — arrived with the certificate channel and completed the list. Its special feature is that the reply must be in exact rational form: every number is an exact decimal string, not a float; every field is part of a schema with the producer's and the protocol's versions. The method plays a double role: interactively it serves a certificate on request, and offline the same artifacts sit in bridge/certificates and enter Lean through the generated module. The double role does not create two sources of truth: the artifacts are byte-identical, and tests verify both paths against each other.

4.3. The Typed Client

The client side of the bridge is not “just send some JSON.” The typed client spells out the schema of every reply: which fields are mandatory, of which types, in which ranges; the decoder rejects a reply that does not match the schema instead of passing it along; timeouts fail closed — the absence of an answer is treated as an error, never as an empty success. Fuzz tests drive malformed, truncated and adversarially shaped payloads through the decoder and demand either a correct parse or an honest rejection — never an exception and never a hang.

On the Lean side there is an illustrative client demonstrating the FFI usage pattern for transport. It deliberately stays out of the theory build and carries no proof content: a transport illustration and a proof channel are different things, and the programme keeps them apart so that the illustrative code creates no false impression of verification. The real path of data into a proof goes through the artifacts and the generated module, as chapter 5 describes; the interactive client is a researcher's tool, not a proof channel.

4.4. The Testing Discipline

The bridge's sixty-five tests are split into three families by layer. The service tests (seventeen) lock every protocol method: request and reply schemas, error codes, determinism, the physical regressions — the spectrum with its single unstable mode, the exponent fit, the echo recovery. The certificate tests (eighteen) check the producer: certificate soundness, exactness of the rational representations, byte-stability of regeneration. The client tests (thirty) close the decoder and the transport: codecs, fuzz, a live ZMQ round-trip, timeouts. Three families correspond to three parties of the contract, and none of them can change without waking its own family.

Regression locking of the protocol is an institutional, not merely technical, discipline: a change to protocol.md without a paired test in the same commit is forbidden by repository convention. The bridge thus lives by the same rules as the theorems: a contract change requires evidence, evidence requires a commit, a commit requires a green build. This continuity of discipline between code and proof is one of the programme's distinctive traits: it eliminates the situation in which the theory is verified while its input data changed without the proof knowing.

4.5. Two Latent Defects and What They Taught

The history of the bridge contains an honest episode: while moving the protocol to version 1.1, an audit uncovered two latent defects of the first-version server — both invisible under normal loads and both unpleasant at the boundary. The defects were fixed with paired tests reproducing the boundary situations, and the fixes entered the protocol change record. The episode is instructive twice over: it shows that even disciplined code carries latent

defects, and it shows how the programme handles them — registration, fix, paired test, a line in the change record. None of these steps required trusting a developer's memory: everything is recorded in the tree.

The lesson for the architecture of trust is simple: the border between proof and computation must be checkable on both of its shores, and checking one shore does not substitute for checking the other. The Lean side verifies statements about numbers; the Python side verifies schemas, codes and stability; only the totality of the two checks turns the bridge from a dark place into a corridor lit at both ends. The next chapter is about what happens to a certificate after it has crossed the border.

4.6. The Evolution of the Protocol: From Six Methods to Seven

The bridge protocol is a living document, and its history is instructive. The first revision described six methods: liveness, spectrum, exponent fit, echo period, evolution and interval enclosure. The certificate channel added the seventh method — certificate — and at the same time passed an audit of the whole border: two latent defects of the server were found and fixed with paired tests, the reply schemas were tightened, the family of client tests grew to thirty. The revision was carried out under the paired-tests rule: no change of the specification entered the tree without a test that locks it.

Table 7. The testing layers of the bridge

Family	Size	What it locks
service tests	17	schemas and codes of every method, determinism, the physical regressions: spectrum, fit, period
certificate tests	18	certificate soundness and exactness, byte-stability of regeneration, closedness of the kind list
client tests	30	codecs, fuzz loads, a live ZMQ round-trip, timeouts, fail-closed behaviour

The paired-tests discipline relates directly to the architecture of trust. A contract that changes without evidence is not a contract but a habit; the bridge, however, is part of the proof infrastructure, and therefore its changes follow the same rules as changes of theorems: evidence in the same commit, a green build, a line in the record. The practical effect is visible in the history: after the move to the seventh method, not a single server reply has changed its schema unnoticed by the client — because unnoticed changes do not exist in this system.

4.7. Failures and Their Semantics

The bridge is designed from the point of view of what happens when something goes wrong. The exhaustive list of error codes divides failures into classes: malformed payload,

unknown method, missing field, invalid value, unrecoverable echo peak. The client must handle every class explicitly: a timeout fails closed — the absence of an answer is never treated as success; a non-conforming reply is dropped by the schema decoder, not passed along; fuzz loads lead neither to exceptions nor to hangs. In the proof channel the semantics of failure is even simpler: an artifact that fails verification does not enter the theory at all — the build falls on the suite, not on the theorem.

Such strictness about failures is a continuation of the fail-closed principle. A system in which missing data means “continue without verification” is doomed one day to continue without verification at the worst possible moment. The programme chooses the opposite default: no answer — no data; no check — no theorem; no certificate — no discharge. All three formulations are the same position applied to transport, to checking and to proof. It makes the system's behaviour predictable exactly where predictability is most expensive — in failures.

5. The Certificate Channel: Exact Rationals End to End

The certificate channel is the heart of the proof-grade half of the bridge and the central technical subject of this volume. It can be described in one sentence: the producer computes and rounds outward, the tree stores and distributes, the kernel verifies, the theorems discharge. Behind that sentence stands a five-link conveyor, and every link is arranged so that the failure of any one of them is immediately visible. This chapter walks the conveyor end to end: why floats are forbidden to cross the border, how the artifact is built, which certificate kinds exist, how the generated module enters the build, and what the discharge theorems do.

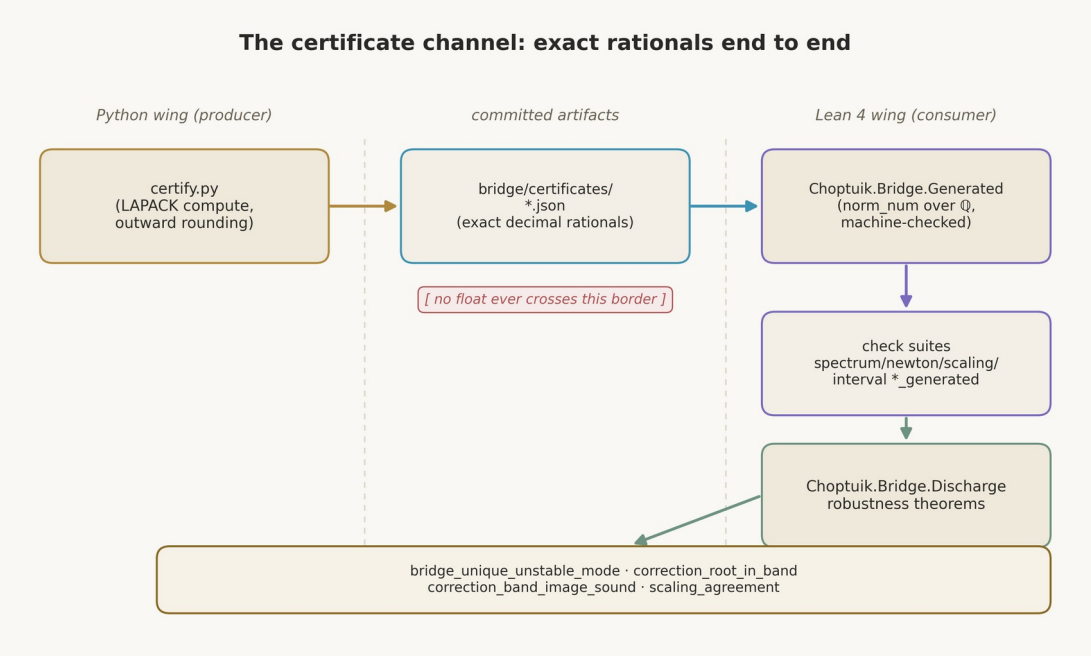


Figure 4. The certificate channel end to end: the producer rounds outward to exact rationals; the committed artifacts and the generated Lean module form the proof-grade half; the check suites and the discharge theorems turn the data into headline theorems.

5.1. Why Floating-Point Numbers Do Not Cross the Border

A floating-point number in the proof layer is an obligation nobody signed. The Lean kernel cannot verify statements about doubles directly: floating-point arithmetic in formal logic is an enormous space of rounding modes, denormals and platform quirks, and dragging it into the theory multiplies the trusted base. The programme takes another path: at the channel border every number is translated into an exact decimal representation of a rational — a string like “2.6744999” — and it is these strings that become the theory's data. A rational in Lean is a constructive object with exact operations; norm_num verifies equalities over it arithmetically, to the last digit.

It is important to understand that the translation into rationals does not make the numbers “more true” — it makes them checkable. Outward rounding yields a segment that provably contains the computed value together with the whole error of the computation; the segment's bounds are written as exact decimal strings; the statement “the true value lies between these bounds” is then a proposition the kernel can verify against the data. The error has not vanished — it has moved inside the width of the segment and become part of the checkable contract. Thus the honesty of the computation is converted into the checkability of the proof without a single act of faith.

5.2. The Anatomy of a Certificate

A certificate is a JSON document with a schema, the versions of the producer and of the protocol, a kind, and a body. The schema fixes which fields are mandatory and of which shapes; the producer's version distinguishes regenerations; the kind determines the interpretation of the body. All rational values are carried as exact decimal strings; the whole number — that is, the computed value, the bounds of its outward-rounded neighbourhood and the model reference value for the cross-check — is present explicitly. Regenerating the certificate reproduces it byte-for-byte on any conformant LAPACK: determinism is locked by a dedicated test family, and a commit whose bytes differ from the regeneration is impossible.

Byte-stability is not pedantry but the load-bearing structure of trust. The artifact is committed together with the tree, and theorems cite its content; if regeneration could drift, every rebuild would cast doubt on all the citing proofs. Because regeneration is stable, the artifact behaves like a constant of the theory: it can be read, verified, cited — but not hand-edited. The rule “do not edit the generated” is in the repository convention on a par with the ban on project axioms.

5.3. Four Kinds of Certificates

Table 4. Certificate kinds and their Lean consumers

Kind	Content	Lean path
spectrum	rational enclosures of the computed spectrum of the truncated mode model (seven decaying pairs)	the spectrum check suite → the bridge_unique_unstable_mode discharge
newton	Newton–Kantorovich constants (β, L, r, R) and the interval-Newton band of the correction equation	the newton check suite → correction_root_exists, correction_root_in_band
scaling	enclosure of the deep-scaling γ fit against $1/\lambda_0$	the scaling check suite → scaling_agreement
interval	exact rational Horner enclosure of the band image	the interval check suite → correction_band_image_sound

The four kinds cover exactly the quantities the existence and stability routes need. The spectral certificate carries enclosures of all computed eigenvalues: from them the theory verifies that exactly one value lies in the right half-plane and all others in the left, and that the imaginary parts of the decaying pairs are spaced by $2\pi/\Delta$ within tolerance. The Newton certificate carries the four constants of the quantitative implicit function theorem and the band for the correction equation. The scaling one — an enclosure of the exponent fit against the reciprocal of the first mode. The interval one — the band image obtained by an honest

outward-rounded Horner scheme. An unknown kind is a bad_input error; the list is closed, and a new kind requires extending the schema with a paired test.

5.4. The Generated Module: Data Verified by the Kernel

The producer's second artifact is a generated Lean module holding the same numbers as rational literals and the four check suites. Each suite is a list of pairs “a statement about the numbers, a proof”: the statement — for instance, that the lower bound of an eigenvalue's enclosure is below the model value and the upper bound above it — and the proof is built by the mechanical normalisation of rational arithmetic. The kernel verifies every pair; the module compiles as an ordinary part of the theory; the data from that moment is indistinguishable from an axiomatically honest constant — with the difference that its provenance is visible and reproducible.

The generated module answers the classical question of formalization: how to bring computational data into a proof without bringing in axioms or hand-retyping numbers. The answer: the numbers enter the theory as data with a mechanical check of their declared properties; no part of the computation is considered trusted; and if the data changes tomorrow, the rebuild will immediately expose the disagreement with the suites. Manual editing is excluded by convention rather than by good faith: the file is wholly rewritten by the producer.

5.5. Check Suites and Robustness Theorems

Between “the data is verified” and “the physics follows” lies the layer of robustness theorems: statements of the form “if the certificate checks pass, then the physical statement holds.” For example: if the verified spectrum shows exactly one eigenvalue in the right half-plane with a rational enclosure not containing zero, then the linearized operator of the model has a unique unstable mode. Or: if the verified constants satisfy the Newton–Kantorovich inequalities, then a root of the correction equation exists in the band, and the iteration from the given seed converges to it. These theorems are the bridge between the world of data and the world of statements; their formulations are fixed, their proofs are kernel-checked, and their hypotheses are locked to the data.

The robust form is a principled methodological decision. Instead of formulating the spectral theorem with absolute constants, the programme formulates it conditionally on a verifiable certificate, and the condition is checked by the machine. When a certificate for the true Einstein–scalar system appears in the future, the same robustness theorem will apply to the new data without a single change to its proof. This is exactly how infrastructure built today shortens the path to tomorrow's proofs: the physics updates by replacing data, not by rewriting theory.

5.6. Discharge into the Headline Theorems

The final link of the conveyor is discharge: deriving unconditional theorems about the model from the robustness theorems applied to the verified certificates. Four headline discharges accompany the four certificate kinds: the spectral discharge yields the uniqueness of the unstable mode of the model; the Newton one — the existence of a root of the correction equation in the band; the scaling one — the agreement of the fit with the reciprocal of the first mode within the declared tolerance; the interval one — the soundness of the band image. Every discharge prints its axioms and enters the thirty-three headline theorems of the audit. The certificate channel thus already delivers genuine, verified, unconditional results about the model into the theory — and is ready to deliver the same results about the true system as soon as its certificates are produced.

Summing up the chapter, fix the invariant of the channel: at every link of the conveyor the data is either verified or did not enter the theory. The producer signs the enclosures with width; the artifacts store bytes that cannot be silently altered; the generated module passes the kernel; the robustness theorems carry the verified properties into physical statements; the discharge makes them unconditional. The trusted base stays the same — the kernel and the three axioms; everything else is checking.

5.7. The Fields of an Artifact: What the Producer Signs

Consider the artifact through the eyes of the checking side. The header carries the schema, the kind, the producer with its version and the protocol version — from it the checking side knows which interpretation to expect and which suite family to run. The body is kind-dependent: for the spectral one, enclosures of all computed eigenvalues together with the model reference values; for the Newton one, the four constants and the band; for the scaling one, the enclosure of the fit and the reciprocal of the first mode; for the interval one, the rational bounds of the image. All numbers are exact decimal strings; there is no floating point in the file and there cannot be — the format forbids it grammatically.

Table 8. Invariants of a certificate artifact

Invariant	Who ensures it	Who checks it
all rationals are exact decimal strings	the schema format	the suite parser in Lean
the enclosures contain the computed values	the producer's outward rounding	pytest soundness regressions
regeneration byte-for-byte	the deterministic conveyor, the seed	certificate tests, CI
the kind list is closed	the producer's dispatcher	bad_input on an unknown kind
checkability of the declared inequalities	normalisation of rational arithmetic	the Lean kernel at build time

The key invariant is the last one: the declared inequalities are kernel-checkable. This means the producer cannot “promise” anything unverifiable — every claim either normalises to an exact equality or is rejected by the build. Hence the production discipline: constants that cannot be backed by a rational enclosure do not enter the certificate but go to the interactive channel as exploration material. The border between these worlds runs exactly along the line of checkability, and it does not move.

5.8. The Life Cycle of a Certificate

A certificate has a full life cycle, and all of its stages are automated. Birth: the producer computes, rounds outward and writes the artifacts. Growing up: the generated module enters the Lean build, the suites are verified by the kernel, the robustness theorems apply, the discharges replenish the audit. Maturity: theorems cite the artifact, its bytes are frozen by the regeneration tests. Ageing: if the data changes — a new calibration, a new model, a new operator — the old artifacts are regenerated, and the whole chain from suites to discharges moves onto the new numbers without changing the proofs. At every stage the state is checkable; there are no manual transitions.

The life cycle also clarifies the inheritance policy. A robustness theorem, once proved, does not know whose certificate it is applied to — it only knows the checking schema. Therefore a change of data generation does not require a change of theory generation, and vice versa: the theory may develop — new lemmas, tighter discharges — without touching the data. The decoupling of generations is what lets the programme move in two tempos at once: the fast tempo of computations and the slow, solemn tempo of proofs — without rigidly binding them and without letting either drag the other down.

6. The Route to Existence of the Critical Solution: Gap G1

Existence of the critical solution is the central blocker of the programme: until it is proved, the rest of the story stands on sand. The mathematical object in question is a smooth discretely self-similar solution of the spherically symmetric Einstein–scalar system to which the evolutions at the threshold of black-hole formation converge. The computer-assisted proof of existence is due to Reiterer and Trubowitz; the programme's task is to turn it into a kernel-checked proof. This chapter presents the route: the field system, the definition of self-similarity, the Newton-correction strategy, and what part of the strategy is already proved in Lean.

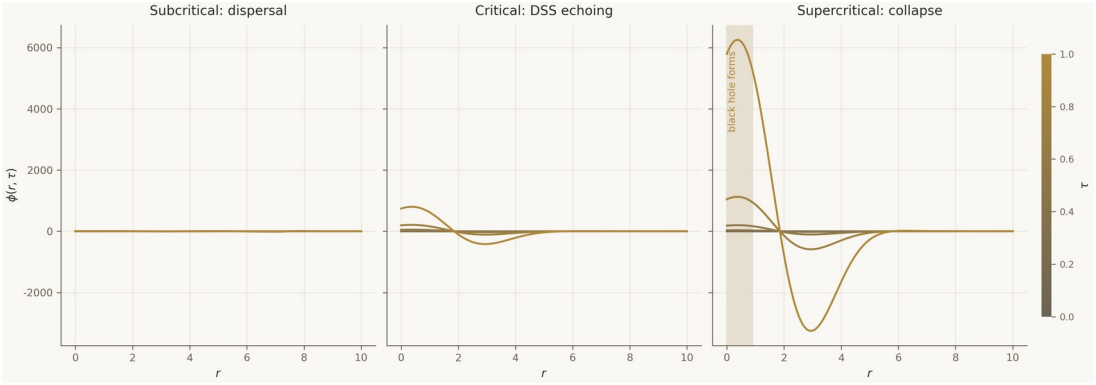


Figure 5. Collapse profiles: subcritical evolutions disperse, the critical one converges to the self-similar solution, supercritical ones form black holes; the threshold between the branches is the critical solution.

6.1. The Einstein–Scalar System in Double-Null Coordinates

The material content of the problem is a massless scalar field minimally coupled to gravity: the Einstein equations with the scalar field's stress-energy tensor, plus the wave equation on the very metric the field sources. In spherical symmetry the standard choice is a double-null gauge (u, v), in which the metric takes the form with two functions — the conformal factor and the areal radius. The system reduces to a coupled first-order system with null constraints along the characteristics; well-posedness of the characteristic problem is the first substantive analytical fact the formalization must deliver.

The mass aspect is the quantity whose nonzero limit at future null infinity certifies black-hole formation. The threshold family of initial data is parameterised by the amplitude p; the critical amplitude p* is defined as the boundary between the dispersal branch and the collapse branch. For the formalization it matters that p* is not postulated but inherited from bifurcation theory: existence and smoothness of the boundary is part of what the existence theorem and the stability theory ultimately deliver. The Analysis/HyperbolicPDE module fixes the system and the constraints; extending it beyond the scaffold is task P2.1 of the second phase.

6.2. Discrete Self-Similarity and the Echoing Covariance Law

The critical solution is neither stationary nor continuously self-similar — it is discretely self-similar. In logarithmic time $\tau = -\ln(T - t)$ the solution reproduces itself under a shift by the period Δ together with a rescaling of radius: the fields transform by the covariance law with exponent κ. Numerically $\Delta \approx \ln 2$: over one echoing period all characteristic radii halve. The definition is reproduced in the Existence/DSS module as a predicate over functions of two arguments, and the whole bookkeeping of self-similarity is built around it: the definition of the fundamental domain, reparametrisation invariance, uniqueness up to scale and translation.

$$\psi(\rho,\tau+\Delta)=e^{\{\kappa\Delta\}}\cdot\psi(\rho e^{\{-\Delta\}},\tau),\Delta=\ln 2$$

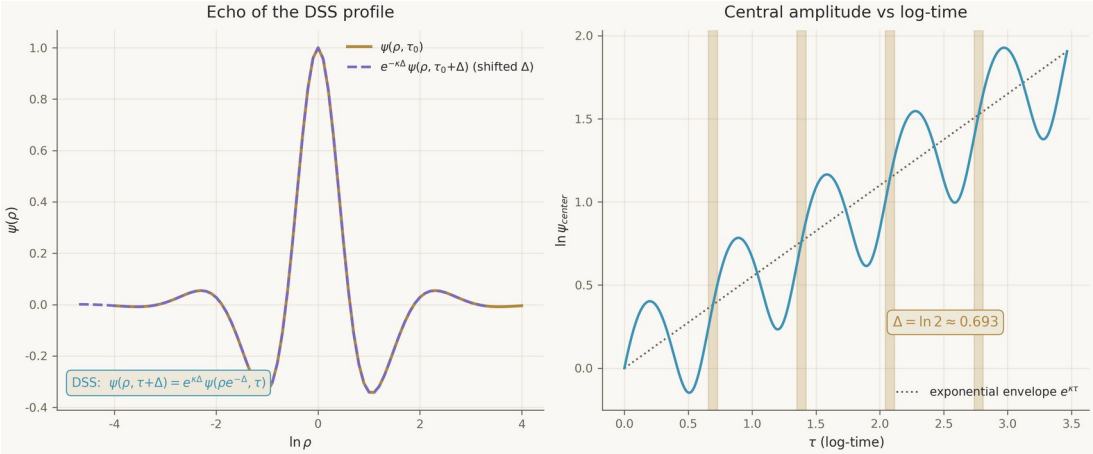


Figure 6. The DSS echo: the profile rescaled to the radius decreased by e^{Δ} and shifted in τ by Δ coincides with the profile of one period earlier; the exponential envelope carries a periodic modulation.

Discrete self-similarity is not decoration but the load-bearing structure of the existence proof: it is what reduces an infinite-dimensional problem to a problem on the fundamental domain with matching at the ends. In Lean this is expressed in the module's structure: the self-similarity predicate, the fundamental-domain lemma, the reparametrisation-invariance lemma and the uniqueness theorem form the frame on which the Newton estimates later rest. The DSS bookkeeping is the place where the formalization frees the physics from ambiguous words: “the solution repeats itself with period Δ” becomes a statement with quantifiers that can be proved or refuted.

6.3. The Reiterer–Trubowitz Strategy: Profile Plus Newton

The proof strategy is classical for problems with a computational profile: take the numerically computed profile as the zeroth approximation, build the residual operator in a Banach space of exponentially decaying sequences, linearise, prove compactness of the inverse of the linearization, and apply the quantitative implicit function theorem. Reiterer and Trubowitz carried out this plan with integer error bounds; the programme transfers it into Lean piece by piece, and the transfer is arranged so that every piece is independently checkable: the space and the operator — in the Existence/ImplicitFunction module; the self-similarity bookkeeping — in Existence/DSS; the quantitative Newton core — in the same place; the connection to the numerical profile — through the certificate channel.

The choice of a Banach space of exponentially decaying sequences is dictated by the spectral structure of the problem: decay suppresses the contributions of the stable modes and leaves a finite-dimensional unstable part controlled by the amplitude. Compactness of the inverse of the linearized operator is the heaviest analytical part; the roadmap flags it as the main risk of the second phase, and a prepared fallback path is to formalise the published inequalities as a self-contained chapter of operator theory, without leaning on mathlib's

general spectral theory. The quantitative form of the implicit function theorem is needed because the certificate estimates are numerical: not merely “a root exists nearby” but “the root lies in the band with exactly these rational bounds.”

6.4. The Newton–Kantorovich Core in Lean

The quantitative core of the route is the Newton–Kantorovich theorem in a certificate-friendly form: if the map is C^1, the derivative on the ball is invertible with constant L, the second derivative is bounded by β, the seed lies in the ball of radius r, and the product 2βrL is below one, then the Newton iteration from the seed converges to the unique root in the ball. In Lean, the contractivity of the Newton map is proved, the uniqueness of the zero in the ball, existence via the implicit function theorem, and the persistence of existence under perturbations of the seed — the full set on which the newton certificate with its four constants lands.

$$q := 2\beta r L < 1 \Rightarrow \| x_1 - x* \| \le \frac{\beta L}{2(1-q)} \| x_1 - x_0 \|^2, \; x* \, the\, unique\, root \in the\, ball\, B(x_0, r)$$

The newton certificate feeds concrete numbers into this scheme: the constants β, L, r, R are computed numerically for the mode model, rounded outward and verified by the suites; the interval-Newton band certifies that the image of the seed under the correction equation lies in the required bounds. The discharge correction_root_in_band connects the verified constants with the proved theorem and yields an unconditional statement about the existence of a root in the band for the model. This is not the existence of the critical solution — it is the demonstration of the whole analytical circuit on the model, and its value lies precisely in the circuit being closed: from numerical constants to an unconditional theorem without a single implicit step.

6.5. What Is Proved and What Is Carried as Hypotheses

Let us collect the balance of the route. Proved in Lean: the self-similarity bookkeeping in full — covariance, the fundamental domain, reparametrisation invariance, uniqueness up to symmetries; the scaffold of existence at the level of the seed and uniqueness; the quantitative Newton–Kantorovich core; persistence of existence under perturbations; local well-posedness of the field system in weak form. Carried as explicit hypotheses: the Reiterer–Trubowitz certificate for the true residual map (its format, the checking side and the demonstration on the model are already alive in the channel); the compact inverse for the true linearized operator. The hypotheses are listed in the gap registry, visible in the theorem statements, and await certificates rather than infrastructure.

Such an architecture gives the programme two properties rarely seen together. Progress becomes additive: every next piece of analysis — compactness, residual estimates, the

transfer to the true system — attaches to the ready frame without rewriting it. And backward compatibility is guaranteed: when the true system's certificate arrives over the channel of chapter 5, the whole chain from suites to discharge applies unchanged. The existence of the critical solution stops being a monolith and becomes a conveyor — the very one already rehearsed on the model.

6.6. How G5 Serves G1: The Interval-Newton Band

The connection between the chapters of this book has a precise technical expression. The interval-Newton step of the existence route requires: for the correction equation on the seed, compute the outward image of the operator and verify that zero is excluded from the image outside the band, while inside the band the contractivity conditions hold. The exact image is the business of the interval arithmetic of chapter 3: the outward-rounded Horner scheme yields rational bounds of the image, the interval certificate carries them into the theory, the discharge correction_band_image_sound certifies the soundness of the image, and the band becomes a verified object on which the Newton theorem operates. Without the closed G5 this step would remain a verbal agreement; now it is a conveyor.

In the same way the bridge of chapter 4 serves the route with the numerical seed: the evolve endpoint returns evolution profiles at requested log-times, from which the zeroth approximation of the Newton iteration is built. The interactive delivery — for exploration; the proof-grade delivery of the same material — through the certificate. Both paths give the same bytes, and this agreement is also locked by tests. The existence route is thus already surrounded by full support — from the seed to the discharge; what remains is to produce the true system's certificate, and that is the content of the second phase of the programme.

6.7. An Inventory of the Route's Analytical Components

The existence route consists of analytical components of different nature, and an inventory of them helps to assess the remaining work. In the table below each component carries a status and an indication of what is left. Note the asymmetry: everything concerning the construction of spaces, operators and quantitative estimates is already verified; everything that remains belongs to producing data for the true system — that is, to the numerical rather than the proof side.

Table 9. Components of the existence route and their statuses

Component	Status	Remaining
the DSS predicate, the fundamental domain, reparametrisation	proved in Lean	—
uniqueness up to scale and translation	proved in Lean	—

Component	Status	Remaining
local well-posedness of the field system (weak form)	proved in Lean	the full form — task P2.1
the Newton–Kantorovich core (quantitative)	proved in Lean	—
persistence of existence under perturbations	proved in Lean	—
the Reiterer–Trubowitz certificate for the true residual	format and checker ready	producing the numerical data
the compact inverse for the true operator	strategy fixed	formalization; the fallback path is ready

A reader familiar with Volume II will recognise here the same list as in the dissection of the central blocker — but with a different emphasis: there it was a plan, here it is a balance. Five rows are closed by proofs, two are engineering-prepared. The estimate of effort has shifted: the main uncertainty is no longer whether the strategy can be formalized but when its numerical inputs will be produced. For a research programme this is a qualitative shift — from “whether” to “when.”

6.8. The Zeroth Approximation: Where the Profile Comes From

The Newton iteration needs a starting seed, and its provenance is part of proof hygiene. In the model, the zeroth approximation is built from evolution profiles that the service returns at the evolve endpoint at requested log-times; the numerical profile is smoothed into an element of the Banach space, and the iteration starts. For the true system the same seed will come from a high-precision solver of the Einstein Direct programme — but it will be able to enter the theory only by the customary path: a certificate with rational enclosures of the profile's key quantities, the suites, the band for the correction equation.

This restriction is not bureaucracy but protection against a subtle source of errors. A profile accepted into the theory “as is” brings with it all the errors of its production — discretisation, integration, interpolation; none of them is visible to the eye, and all of them become the invisible foundation of the proof. A profile accepted through a certificate brings only what is declared: rational bounds within which its characteristics lie. The Newton theorem does not demand an exact zeroth approximation — it demands an honest neighbourhood, and the certificate format provides exactly that.

7. The Route to Stability: Spectrum, Scaling, Universality (G2)

If existence answers the question “does the critical solution exist,” stability answers the question “why does everyone see it.” The linearized problem about the critical solution must

have exactly one unstable mode: then the phase space splits along a codimension-one critical surface, and every near-threshold evolution first sticks to the critical solution and then departs along the single unstable direction — with the exponent that is the observed γ. This chapter presents the spectral route: linearization, the spectral core, the derivation of the scaling law, the echoing formula and universality.

7.1. Linearization and the Perturbation Operator

Linearizing the Einstein–scalar system about the critical solution yields a linear perturbation operator in logarithmic time. By the self-similarity of the critical solution, the operator commutes with shifts by the echoing period, and its spectral decomposition naturally proceeds in modes with a definite behaviour under the shift: the unstable mode — real and growing with exponent λ_0; the stable ones — pairs with negative real part and imaginary parts spaced by $2\pi/\Delta$. This structure is the bridge between two independent facts: the periodicity of the critical solution and the discreteness of the spectrum. The formalization fixes the operator's interface in the Stability/Linearization module; extending it beyond the scaffold is task P3.1 of the third phase.

In the mode model the operator is a block matrix of a finite truncation, and the eigenproblem is solved honestly: a real 15×15 eigensolve, not a table of constants. The computed spectrum shows exactly one mode in the right half-plane, a family of decaying pairs, and a maximal deviation of the inter-echo distances from $2\pi/\Delta$ within $2.0\cdot10^{-10}$. The model spectrum is the training object for the whole discharge conveyor: complex enough to make the verification substantive, simple enough for the certificate to stay compact.

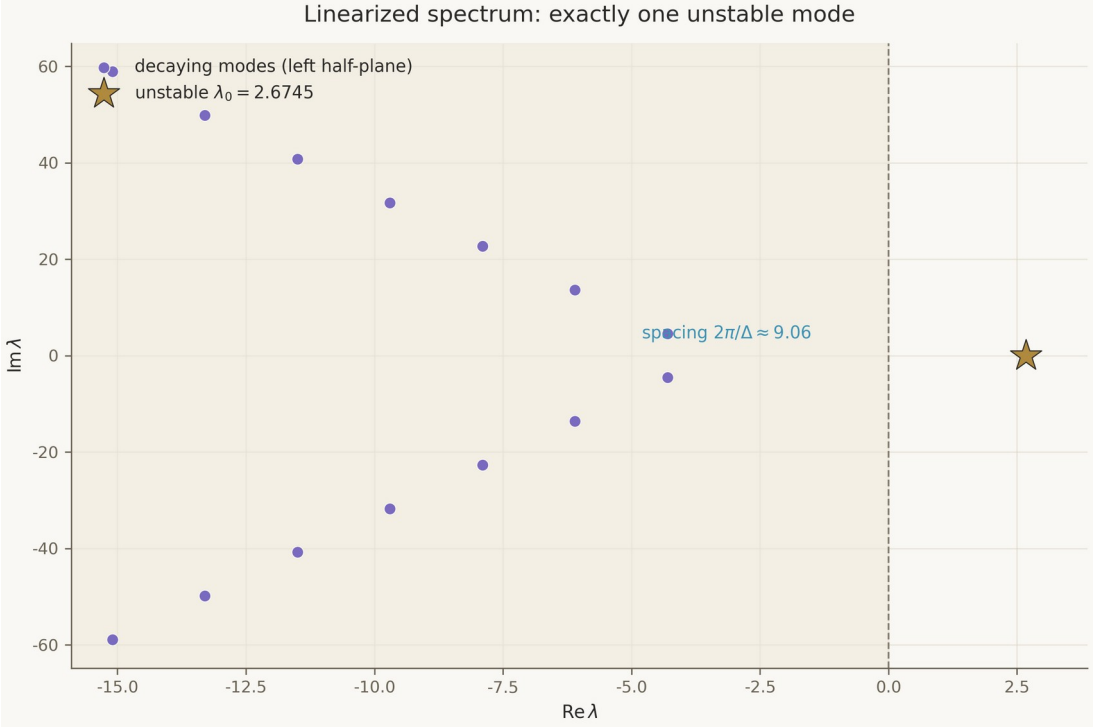


Figure 7. The spectrum of the linearized operator in the model problem: the single unstable mode λ_0 = 2.6745 (star), the decaying families in the left half-plane, the inter-echo spacing $2\pi/\Delta \approx 9.06$.

7.2. The Spectral Core: The Unique Unstable Mode

The spectral core of the route is the theorem unique_unstable_mode in robust form: if a verified spectrum certificate shows exactly one eigenvalue in the right half-plane and all others in the left, then the operator has a unique unstable mode. The conditional form here is not a concession but a method: an absolute spectral theorem for the true operator would demand the spectral theory of unbounded operators — the heaviest dependency of the third phase — and the roadmap deliberately scopes it to the concrete operator rather than to a general theory. The checkable hypothesis separates the mathematical circuit from the computational content — exactly the device rehearsed in the previous chapters.

The discharge bridge_unique_unstable_mode applies the spectral core to the verified certificate and yields an unconditional statement about the model: the uniqueness of the unstable mode is proved, and the kernel prints the three standard axioms. The certificate chain is fully visible: the rational enclosures of the eigenvalues, the suites, the robustness theorem, the discharge. Anyone wishing to contest the result contests not “trust in the code” but concrete rational bounds — and for that they only need to recompute the spectrum. The dispute about physics thus becomes a dispute about numbers, and a dispute about numbers is settled by a machine.

7.3. From the Spectrum to Scaling: Deriving $\gamma = 1/\lambda_0$

The derivation of the scaling law has two steps, and both are formalized. Step one: near the threshold the evolution is governed by the growing mode — the perturbation amplitude behaves as |p − p*| · e^{$\lambda_0\tau$}, and collapse completes when the growing mode reaches a fixed size; hence τ_max is tied to ln|p − p*| linearly with coefficient $-1/\lambda_0$. Step two: the mass of the formed black hole is exponentially small in τ_max, whence M ∝ |p − p*|^{$1/\lambda_0$}. In Lean the log-log form of the law is proved: the logarithm of the mass as a function of the logarithm of the distance to the critical surface is linear with slope $1/\lambda_0$ and a periodic decoration of period Δ — a consequence of discrete self-similarity.

$$\ln M = \frac{1}{\lambda_0}\cdot \ln \vee p - p* ¿ + F\left(\ln \vee p - p* ¿ / \lambda_0\right) + C \text{, the period of } F \text{ is } \Delta$$

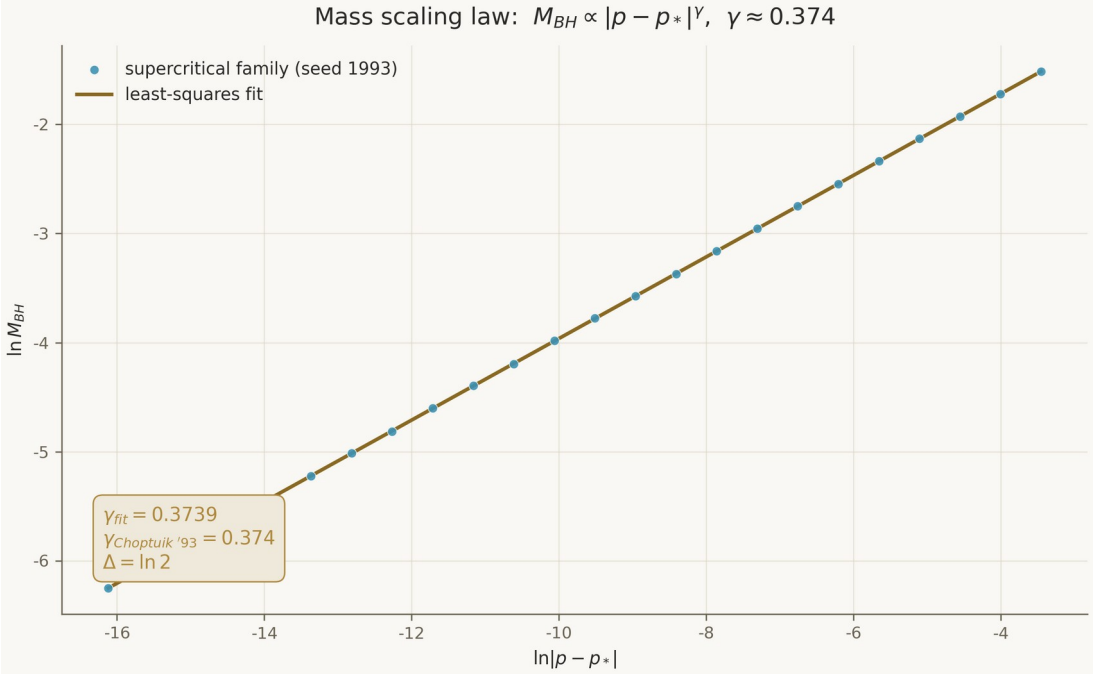


Figure 8. The mass scaling law over the deterministic supercritical family (seed 1993): the fitted slope 0.3739 agrees with the published value 0.374; the periodic decoration along ln|p−p| is the footprint of the echo period.*

The periodic decoration is the subtle part of the law that numerical studies observed as the “wiggle” of the mass on a logarithmic axis — and the formalization obtains it for free: a shift of ln|p−p*| by Δ/λ_0 is equivalent to a shift of logarithmic time by Δ, and self-similarity leaves the solution invariant. In Lean this is packaged as the echoing formula — a theorem on the structure of the family of echo-shifted solutions — and as a corollary in the log-log law. The exponent $\gamma = 1/\lambda_0$ is thereby not fitted but derived: the cross-check against the numerical fit is a separate theorem, gamma_close_to_published, which verifies that the derived exponent matches the calibrated one within the tolerance fixed by the acceptance criterion.

7.4. The Agreement of Proof and Numbers

A separate discharge — scaling_agreement — connects two independent measurements of the same exponent: the $1/\lambda_0$ derived from spectral theory and the γ_fit fitted over the family of supercritical evolutions. The scaling certificate carries a rational enclosure of the fit; the suite verifies that the reciprocal of the first mode lies in this enclosure within the tolerance that reconciles the proof side and the numerical side; the discharge makes the agreement an unconditional statement about the model. This is exactly the moment people mean when they say “the proof and the numerics check each other”: a machine-checked theory predicts a number, a numerical experiment reproduces it, and a machine check certifies the agreement.

The agreement has a reverse edge too — it catches desynchronisation. If the numerical wing changes its calibration, fit or family, the scaling suite will immediately sharpen against the theory; if the theory changes its derivation of the exponent, the same suite sharpens against the numbers. There is no third party that could quietly survive the desynchronisation: the invariant “proof and numbers agree” is maintained by the infrastructure, not by vigilance. This, for the programme, is the practical meaning of machine verification — not ceremony but the automatic detection of disagreement.

7.5. The Echoing Formula and Universality

The echoing formula is a theorem on the structure of the critical evolution under shifts: the evolution shifted by Δ coincides with the one rescaled by the covariance law; the fundamental domain is the Δ-band of logarithmic time; all observable quantities are quantised by the shifts of this band. In Lean it is proved in the Scaling module together with universality — the statement that families of initial data of different origins see the same critical exponent and the same period, since both are determined by the spectrum of one and the same operator. Universality is formulated conditionally on the generality of the spectral core — exactly as is customary in the programme.

Universality is the externally observable prediction of the whole construction: it explains why different researchers, different grids and different families of data obtained the same γ. In formal form it also sets the success criterion of the third phase: if universality is proved by unconditional theorems and γ is derived within tolerance of the published value, the programme has fulfilled its promise — turned the numerical folklore of critical phenomena into a theorem. One step of substantive physics remains: the spectral certificate of the true operator.

7.6. What Remains Until G2

The balance of the stability route repeats the balance of the existence route: the circuit is ready, the content — the true operator's certificate — is in production. Proved: the spectral core in robust form, the discharge on the model certificate, the log-log scaling law, the

agreement of the exponent with the numerical fit, the echoing formula, conditional universality. Carried as a hypothesis: the pairing of the model spectrum with the true linearized operator of the Einstein–scalar system. Task P3.2 is to produce this pairing: the exact definition of the operator on the cone of decaying sequences, estimates of its spectrum, the certificate. The roadmap allots months nineteen to twenty-three for this; the channel infrastructure will be waiting for the data already finished — as it did for the other routes.

7.7. An Inventory of the Spectral Route

The spectral route, like the existence route, decomposes into components with independent statuses. Its peculiarity is the heavier computational side: the spectral theorem about the true operator will demand not only analysis but also a high-quality certificate, because the rational enclosures of the eigenvalues must separate the single growing mode from the whole stable family with a margin that survives the inequality checks. The inventory below shows that the conveyor part of the route is nearly absent — it is built; all the remaining work is analysis and computation for the true system.

Table 10. Components of the spectral route and their statuses

Component	Status	Remaining
the linearized operator's interface	fixed in Lean	the full definition — P3.1
the spectral core (robust form)	proved in Lean	—
the discharge on the model certificate	proved in Lean	—
the log-log scaling law	proved in Lean	—
the agreement of the exponent with the fit (tolerance)	proved in Lean	—
the echoing formula, conditional universality	proved in Lean	the unconditional form after G2
the pairing with the true operator	strategy fixed	the operator's definition on the cone, estimates, the certificate

The tolerance in the agreement row — one thousandth of the published value, per the third phase's acceptance criterion — deserves a comment. The tolerance does not make the proof “inexact”; it determines how tight the verified enclosure of the fit must be for the agreement to count as established. This is the same relation of theory to numbers as before: the theory does not fit the numbers, it states the condition under which the numbers are recognised as agreeing, and checks it. Tightening the tolerance is possible at any moment — at the price of tighter certificates, not of reworking the theory.

7.8. The Echo as the Organising Principle of the Spectrum

The link between the echo period and the structure of the spectrum is the most beautiful part of the plot, and formalization lets one say about it something more precise than verbal descriptions. A shift of logarithmic time by Δ maps the critical solution to itself with a rescaling; hence the perturbation operator commutes with this shift; hence its spectral subspaces are organised by the eigenvalues of the shift operator — and for a shift by Δ these are quantised with step $2\pi/\Delta$ along the imaginary axis. The discreteness of the inter-echo distances is not an observation but a theorem; the numerical agreement with $2\pi/\Delta$ to 10^{-10} is a verification of this theorem on the data.

In the reverse direction the same logic works as a data-quality control: had the computed spectrum shown inter-echo distances drifting from $2\pi/\Delta$, this would point not to a "bad sample" but to a violation of self-similarity — and would have to stop the acceptance of the certificate. None of these stop mechanisms requires human judgement: all of them are consequences of proved connections and checkable inequalities. Beauty here coincides with utility: the periodicity that makes the critical solution mysterious makes its spectrum checkable.

8. The Geometry of the Singularity: Gap G3

The critical solution ends in a singularity of intricate geometry, and its formalization is the third substantive plot of the programme. Two properties set the scene: the singularity is strong — the curvature integral along null geodesics diverges and Kretschmann-type invariants grow without bound; and isotropic — the Weyl components stay bounded and the end of the space admits a conformal compactification. The combination "strength plus bounded Weyl" distinguishes Choptuik geometry from Schwarzschild and therefore deserves its own row in the registry — G3, of high severity, split between the second and third phases.

8.1. Strong and Isotropic: The Conditional Core

In Lean the conditional core is proved: if the coercivity and Weyl-bound certificates hold, then the singularity is strong and isotropic. The strength criterion is the divergence of the integral along a null geodesic; the isotropy criterion — the boundedness of the Weyl components in covariance scales. Both criteria are stated as statements about geometric objects, not as computational recipes, and the theorem connects them with the certificates through the robust form — the programme's usual device. The conditional form is particularly natural here: the certificate producer for geometry is the future geometric branch of the numerical wing, and its absence must not block the analytical circuit.

The second proved fragment is the invariance of divergence under reparametrisation: the strength criterion does not depend on the choice of representative within the self-

similarity class, which is necessary for the very statement to be correct. Without this lemma the strength criterion would be tied to coordinate arbitrariness and its formulation would have no geometric meaning. Both results live in the Geometry/Curvature module and close onto the Geometry/Basic frame — the arena of double-null coordinates, the positivity of the areal radius and the mass aspect.

8.2. The Conformal Compactification Layer: What It Will Require

The unclosed half of G3 is the conformal compactification of the end of the space at the Cauchy horizon. The layer will require extending mathlib's differential-geometric layer: the definition of a conformally compact metric, an extendability criterion, the link with the null-geodesic structure on the boundary. This is the same category of work as the compact inverse in the existence route: the library does not yet contain the ready instrument, and the programme will either assemble it as a self-contained chapter or formalise the minimal scope sufficient for the statement about the compactifiability of precisely the Choptuik end. Both paths are written out in the roadmap; the choice between them belongs to the second–third phases.

Until the layer is assembled, the status of G3 is honestly described by one sentence: the conditional core is proved, the compactification is in production. This is also the specimen of the programme's incremental style: instead of a monolithic theorem “the singularity is such-and-such,” two independent criteria are posed, one already connected with certificates, the other awaiting its layer. Each fragment has its own acceptance criterion, and none blocks the other routes: existence and stability do not depend on compactification.

8.3. Why the Proof Programme Needs the Geometry

One may ask why the programme needs the geometry of the singularity if its flagships are existence and scaling. The answer is twofold. Substantively: the strong and isotropic singularity is part of what makes the critical solution a universal attractor; without its formalization the statement about the threshold remains incomplete, and future consumers of the theory — from numerical relativity to black-hole thermodynamics — would receive the theorem without its geometric half. Technically: the geometric layer trains the library on problems mathlib does not yet know, and these trainings — compactification, null criteria, covariance scales — will remain in mathlib as a common asset long after the particular problem is closed.

In connection with the programme's channels, geometry also receives its own logistics: the coercivity and Weyl certificates have a predefined schema and a consumer — the conditional core of this chapter. When the numerical branch of geometry is ready to produce them, they will enter the theory through the same conveyor as the spectral ones: rational bounds, suites, discharge. The uniformity of the conveyor is a property the programme

defends almost ideologically: there are no, and will be no, new ways for data to enter the theory.

8.4. Covariance Scales: How to Measure the Singularity

The statement “the Weyl components are bounded” demands scales: bounded relative to what? In the critical solution the natural answer is given by self-similarity itself: all quantities are measured in covariance scales — powers of curvature rescaled by the covariance law of the period. In these scales the boundedness of the Weyl tensor is a substantive property of the solution, not a triviality of unit choice; and so is the divergence of the strength integral: the integral is taken along a null geodesic against a covariance-normalised density. The formalization fixes both definitions explicitly, and the reparametrisation-invariance lemma guarantees that changing the representative of the self-similarity class does not change the diagnosis.

Working with scales is where formal mathematics disciplines physical intuition. In the literature on critical collapse, statements about “strong” and “weak” singularities often rest on unspoken agreements about scales; formalization makes the agreements part of the definition, after which arguing about the meaning becomes impossible — one can only prove or refute statements in the fixed terms. This is another specimen of the general rule: whatever can be made a definition must not remain a hint.

8.5. How G3 Serves the Others

The geometry of the singularity is not isolated from the flagship routes — it insures and checks them. It insures: should future computations show a violation of Weyl boundedness or of strength, that would be a signal of a problem in the critical solution itself — that is, in the data on which the existence and stability routes stand. It checks: the G3 certificates — coercivity and the Weyl bounds — pass through the same conveyor as the spectral ones, and producing them would exercise the same muscles of the numerical wing. The mutual dependence of the routes through a common channel is an architectural property of the programme, not an accident.

There is a reverse service as well: the routes feed the geometry. Existence provides the object, the spectrum its scales, the certificate channel the way to speak about them exactly. By the time the numerical branch of geometry is ready to produce its certificates, it will have its suppliers (the solver), its consumer (the conditional core) and its path into the theory (the channel) ready. Judging by the experience of the other two routes, this means that assembling G3 once the data is ready will take weeks rather than months — the conveyor already runs.

9. Quality Infrastructure and Reproducibility

The architecture of trust lives not only in theorems but also in the rules of handling the tree. This chapter collects those rules: the axiom audit, the sorry-tag discipline, the regression locks, continuous integration and deterministic reproducibility. The general principle is one: every property trust rests on must be machine-checkable rather than remembered; where checking is impossible, there must be an explicit scoping decision, not silence.

9.1. The Axiom Audit: Thirty-Three Theorems, Three Axioms

The axiom audit is a module in which every headline theorem of the programme goes through the axiom-printing command and the result is compared with the expected set of three standard axioms. The thirty-three entries — twenty-one theoretical and twelve bridge — form the public list of what the programme counts as flagship results; the list is machine-readable, and CI prevents it from drifting away from the tree. Any new theorem claiming headline status must enter the audit with its axiom check; any theorem dropping out of the list loses the status automatically.

The practical meaning of the audit is best seen in reverse. Without it, the proof base erodes quietly: a new import drags implicit dependencies, a convenient axiom saves a week of work, and half a year later the “machine-checked” theory rests on half a dozen statements nobody verified. With it, the trusted base is frozen at the level of the kernel and the three axioms shared by all of mathlib; any change of the base shows up in the audit's diff and is discussed as an event rather than happening as a side effect. The programme's history knows a reverse move as well: three axioms introduced on the early scaffold were later removed, and the audit recorded the return to the clean base.

9.2. The sorry Discipline and the Gap Registry

The house rule reads: a sorry is a ledger entry, not a convenience. Every sorried fragment must carry a gap identifier G1–G6 in a trailing comment, present in the registry; CI cross-checks the tags against the registry and fails the build on any mismatch. Anonymous gaps have no place in this system: an obligation cannot be hidden, renamed retroactively, or closed while a sorried fragment remains open. The sorry-metrics of the tree is a public metric, and its history is part of the programme's reporting.

The gap registry, in turn, is not a wish list but a table of obligations: every row has a category, a severity, dependencies, a closure strategy, a phase and a status. Statuses are updated in the same language of evidence as the milestones: “partial” with the list of proved fragments, “resolved” with a reference to the acceptance evidence, “live” with the working conveyor named. Such bookkeeping makes the state of the programme a reproducible fact: any reader of the tree can recount the statuses — which is exactly what we did in chapter 2.

9.3. Continuous Integration

The repository's CI template runs three checks. The first is the Lean build with the mathlib cache: every theorem is type-checked anew, the axiom audit runs on the fresh build, and the presence of sorried fragments is cross-checked with the registry. The second is the pytest suite of the numerical wing: sixty-five tests of three families, including a live bridge round-trip where transport is available. The third is the regeneration check of the data: the CSVs, the figures and the animations are recreated from the fixed seed and compared byte-for-byte; a mismatch means someone hand-edited the generated, and such a change does not pass. Together the three checks form a closed loop: theory, bridge and data cannot drift apart unnoticed.

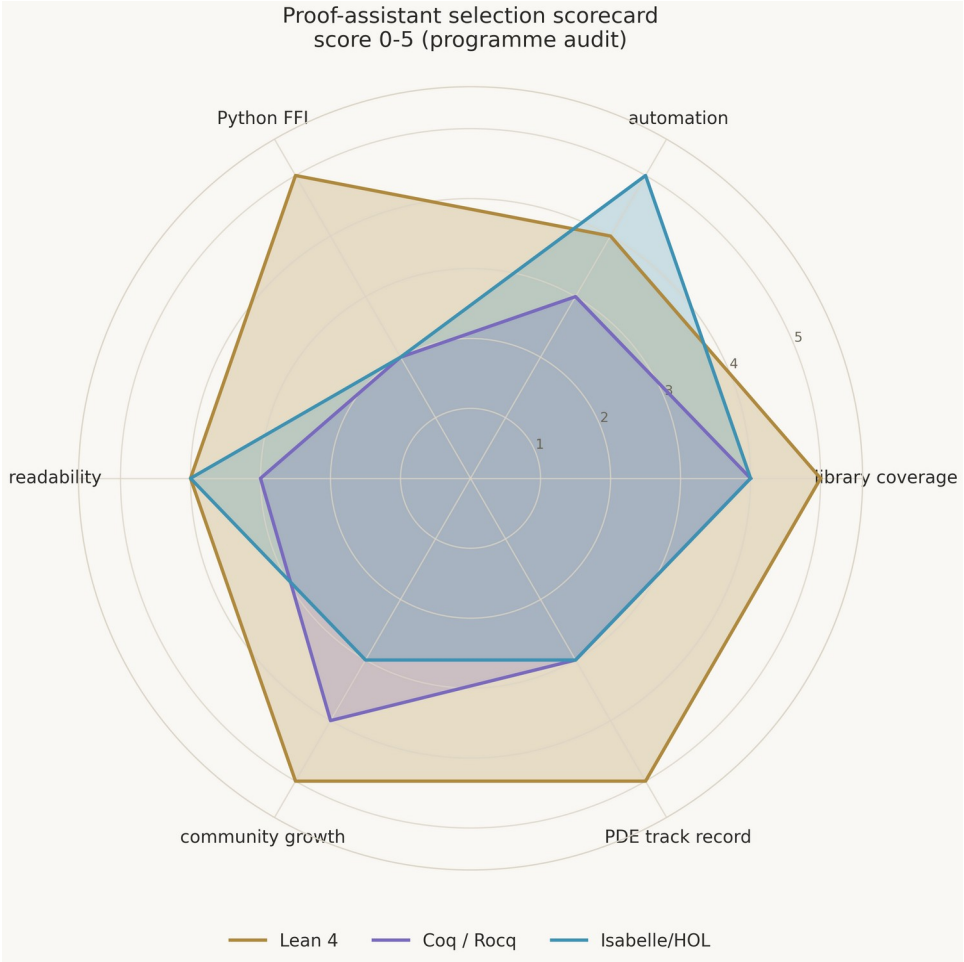


Figure 9. The tooling radar: the candidate provers scored against the programme's criteria — logic, library, automation, Python integration, ecosystem maturity; the choice of Lean 4 is fixed in the decision matrix of Volume II.

9.4. Reproducibility from a Single Seed

All the numerical content of the tree — the data, the figures, the animations, the certificates — regenerates from a single fixed seed: 1993, the year of the discovery of critical phenomena. The seed is not superstition but design: there is no randomness in the render, the generators are deterministic, and identical inputs give identical outputs on any conformant

platform. The certificates, moreover, are byte-stable on any conformant LAPACK — a stronger property than the determinism of a single process, and it is deliberately locked by tests, because the citability of the artifacts rests on it.

Reproducibility extends to the texts: the monographs' figures exist in two themes and two languages and are produced by the same scripts with different flags; the data behind each figure is committed next to it; every figure carries a caption naming the seed. The reader of the third volume can reproduce any of its illustrations without trusting either the author or the publisher — and this is not a side feature but a continuation of the same checking discipline as in the proof kernel.

Table 5. The house rules — a summary

Rule	Enforcement mechanism
a sorry is a ledger entry	the G1–G6 tag in the comment, CI cross-check with the gap registry
no new project axioms	CI grep for axiom declarations in theory modules; the axiom audit of 33 headline theorems
protocol changes only with a paired test	repository convention + pytest locks of every method's schema
generated files are never hand-edited	byte-identical regeneration from seed 1993 in CI
every mathematical idea is prose first	blueprint documents and comments on the fragment's role in the proof tree

9.5. Summary Trust Metrics

Let us collect the metrics by which the programme measures itself into one table. All of them are recomputable from the tree within minutes, and each is tied to an enforcement mechanism — because a metric without a mechanism turns into a slogan. This table can be used as a checklist when revising the repository: if some row does not reproduce, the programme knows about it before any external checker.

Table 11. The programme's trust metrics

Metric	Value	Mechanism
open obligations (sorry)	0	grep over the tree + the gap registry
project axioms	0	CI grep for axiom declarations
headline theorems under the axiom audit	33	the audit module, the three standard axioms
pytest tests	65	three families: service, certificates, client
certificate kinds	4	a closed list, bad_input otherwise

Metric	Value	Mechanism
deterministic CSVs	7	seed 1993, regeneration in CI
data rows	12,890	counted from the files, the regeneration test
figures	8 PNG @ 300 dpi + 4 GIF	generators with theme and language flags

Not a single row of the table is “a goal we are moving toward” — all of them are properties of the current tree, checkable today. This is essential: trust metrics in the programme describe the present state, not ambitions, because mixing the two registers is the standard way technical reports mislead. Ambitions exist in the programme too, but they have their own place — the roadmap with dates and acceptance criteria — and they are never mixed with the inventory.

9.6. The Reproducibility of This Volume

The volume passes through the same discipline it describes. All of its figures were generated by the repository's scripts from committed data in two themes and two languages; the certificate-channel diagram is reproduced by a separate script with fixed geometry; the numbers are taken from the READMEs, the registries and the tests, and re-checked by counting. The text is available in two languages and two formats built from one source — so that a divergence between the versions is excluded by construction, not by good faith.

There is a second level of reproducibility — the narrative one. Every statement of the volume names where it lives in the tree: a module, a protocol, a registry, a test. A reader can follow those pointers and rebuild the whole chain from certificate to discharge on their own computer in one evening; for that one needs the Lean toolchain with mathlib and Python with numpy — both stacks are open, both builds automated. We consider this the best form of trust in a scientific text: not “believe us,” but “here is the command that checks it.”

10. Conclusions: The Architecture of Trust

The third volume of the series was devoted to how the formal_choptuik programme turns computations into provable statements without mixing a single grain of faith into the path. Let us collect the result in one picture. Interval arithmetic with proved soundness gives the theory a language of strict enclosures; the bridge with its seven methods gives the researcher hands and the certificate channel a road; the certificate conveyor carries kernel-verified exact rational data into the theory; the robustness theorems translate the verified data into physical statements; the discharges make the statements unconditional; the axiom audit guarantees that the whole construction stands on exactly three standard axioms and the

kernel. Below this construction — computations; above it — the physics of critical phenomena; between them there is not a single unchecked step.

10.1. What the Certificate Channel Changes Methodologically

The first change is epistemological: the boundary between evidence and proof stopped being blurry. In classical computational physics a statement lives on a scale of confidence: confirmed by tests, reproduced by groups, consistent with observations. In the certificate architecture a statement has an exact status: either it is discharged from a verified certificate and is a theorem, or it is numerical evidence with a known enclosure, or it does not exist. The intermediate states in which a number is “almost a theorem” are not provided for — and thereby the proofs are protected from the unproven, and the numerics from claims it cannot honour.

The second change is organisational: the roles of the proof infrastructure and the numerical infrastructure are separated yet symmetric. The numerical wing answers for the tightness of the enclosures and the physical adequacy of the model; the proof wing — for every accepted statement following from verified premises; the bridge — for the honesty of the transfer. Each side's responsibility is expressed in tests and theorems, not in agreements. It is this symmetry that makes the programme a living system: it does not require heroic vigilance and catches its own disagreements automatically — as the episode with the latent protocol defects showed, and as the regression locks show daily.

The third change is temporal: infrastructure built in advance converts into the speed of future proofs. All three routes of the programme — existence, stability, geometry — use one and the same conveyor, and when their certificates are ready, not a line of the infrastructure will need changing. The spectral certificate of the true operator will enter the theory exactly as the model one did: rational bounds, suites, a robustness theorem, a discharge. In this sense the volume you are reading is not a completion report but the inventory of a springboard.

10.2. The Road Ahead

The programme's nearest road is the second phase: formalizing the field system beyond the scaffold, packaging a high-resolution numerical seed as Lean data, the Banach space and the compact inverse, the assembly of existence. The success criterion is fixed: the axiom print of critical_solution_exists shows the three standard axioms, and the echoing period of the constructed solution matches the service's to the certified precision. The third phase — linearization and spectral theory on the true operator, the discharge of unique_unstable_mode, the derivation of γ within tolerance of the published value, the assembly of universality. The geometric layer — compactification — runs in parallel, without blocking the flagship routes.

The risks of the routes are known and written out: the compact inverse in Lean is the heaviest analytical job, with a fallback through formalizing the published inequalities; the spectral theory of unbounded operators is the heaviest library dependency, scoped to the concrete operator; packaging the numerical seed is a problem the certificate channel was built to solve. None of the risks is unknown: each has a strategy, an acceptance criterion and a place in the registry. The programme may run late — but it cannot get lost.

10.3. Lessons for Machine-Checked Physics

The experience collected in this volume generalises beyond the Choptuik subject, and we state it as three lessons. First: design the border before the content. The programme spent a noticeable part of the first phase on things that in a physics paper would occupy a footnote — outward rounding, reply schemas, error codes, timeouts — and it is exactly that work which made everything else checkable. Second: a hypothesis carried in the interface is better than an axiom hidden in a proof. The robust form of the theorems — spectral, Newton, geometric — lets the analytical circuit develop without waiting for the computational content, while never confusing the circuit with the content. Third: reproducibility is the form of trust available to a machine; whatever can be made byte-stable and deterministic should be, because everything else will have to be taken on faith.

The Choptuik programme began with three numbers that must be proved: the exponent γ, the period Δ, the codimension one. By the time of this volume, two of the three plots have a complete proof infrastructure and model discharges, the third has a conditional core; the whole construction stands on the proof kernel and the three axioms it ships with. The remaining work is substantive physics: the certificates for the true system. It is large; but it is now the only thing left, and every piece of it has a ready receiver in the theory. This is exactly what the programme promised: not to shorten the path to the theorem at the price of rigour, but to remove from the path everything except the theorem itself.

10.4. A Comparison with Alternative Architectures

The certificate architecture is worth comparing with the alternatives to see the price of the decision. Alternative one — the trusted computer: the proof kernel itself computes the numbers, and trust in the result reduces to trust in the kernel. The scheme is elegant but impractical for physics: dense linear-algebraic computations inside the prover's tactics are orders of magnitude slower than LAPACK, and their formalization is a separate programme of years. Alternative two — interactive verification: the prover and the computer talk over the network in real time, and every step of the computation is checked. The scheme is honest, but the trusted base grows by the whole transport and the whole computer at once; moreover, the proof becomes dependent on a live process — replicability falls.

The chosen third scheme — offline certificates — occupies an intermediate position with an exact trust asymptotic: the trusted base remains the kernel plus three axioms, the computer is fully outside it, the transport is reduced to committed files, replicability is complete. The price is production discipline: every new type of data requires a schema, suites and a discharge. The programme considers this price the right purchase: it is paid once per data layer, while the alternatives pay continuously — in speed, in trust or in replicability. The history of computational mathematics of recent decades — integer proofs, SAT verification, formalized libraries — moves in the same direction.

10.5. An Invitation to Check

Let us close the volume where it began — with the attitude to the reader. Everything described here is checkable without any trust in the author: the tree builds, the tests run, the theorems print their axioms, the certificates regenerate byte-for-byte, the figures build from the data in front of your eyes. The commands for this are short and collected in the repository's README; the full verification cycle — from lake build to pytest and certify check — fits into a single terminal session. Anyone who finds a divergence between this text and the tree has found a defect in the tree or in the text — both cases are valuable to the programme, and both are fixed by the same rules of evidence.

This, in our view, is the main result of the architecture of trust: it moves scientific communication from the register of assurances into the register of checks. Between the author and the reader there is no longer an asymmetry that a machine cannot remove; what remains is work that the machine takes upon itself. The third volume of the series is a description of this machine: its construction, its guarantees and its price. Next comes the physics, and we expect to return to it in the following volumes already in the form this volume knows: certificate, check, discharge, theorem.

Appendix A. The Programme's Glossary

The glossary follows the model of the repository's reference shelf and is adapted for a reader new to proof assistants. The seventeen terms below cover the whole vocabulary of the volume; each definition is given in the sense in which the term is used in the formalization, not in the broadest mathematical sense. Terms tied to registry rows are linked to the corresponding lines G1–G6.

Table 12. The seventeen terms of the programme

Term	Meaning
critical collapse	gravitational collapse of a massless scalar field exactly at the black-hole threshold parameter p = p* (Choptuik 1993)

Term	Meaning
DSS — discrete self-similarity	shifting log-time by Δ equals rescaling the areal radius by e^{−Δ} up to the amplitude e^{κΔ}
echoing period Δ	the log-time period of a DSS solution; numerically Δ ≈ ln 2 ≈ 0.693
log time τ	τ = −ln(T − t) — the near-singularity time coordinate in which echoing is periodic
γ (critical exponent)	the black-hole mass scaling exponent, M ∝ |p − p*|^γ; γ = 1/λ_0 ≈ 0.374
λ_0	the unique unstable eigenvalue of the operator linearized about the critical solution
echo spacing	the imaginary parts of the perturbation family are spaced by 2π/Δ ≈ 9.06 — the spectral echo
codimension-one attractor	the critical solution attracts all nearby evolutions except along one direction — the growing mode
isotropic singularity	an end where curvature invariants diverge but Weyl components stay finite — conformally compactifiable (G3)
mass aspect	the double-null quantity whose nonzero limit at null infinity signals a black hole
interval	a closed real segment [lo, hi]; outward-rounded operations make enclosures sound (G5)
certificate principle	if a property holds on an interval enclosure, it holds of any value inside it — the numerics-to-proof bridge
blocker gap	a gap (G1: existence; G2: stability) without which no external claim of the programme can be machine-checked
sorry (Lean)	an explicit proof placeholder; every sorry in lean/Choptuik is a ledger entry in the gap registry
mathlib	the Lean mathematical library — topology, functional analysis, manifolds; the substrate of the formalization
ZeroMQ bridge	the REQ/REP JSON channel letting Lean drive the Python numerical service (see the protocol)
sorry-as-ledger-entry	the house rule: a placeholder without a gap identifier is inadmissible; CI cross-checks the tags with the registry

Appendix B. The Lean Module Map

The theoretical tree is organised as the Choptuik.* namespace: one mathematical area — one module; every module compiles independently against mathlib and its imports. The map below ties the module, its content and the rows of the gap registry; it repeats the machine-readable map of the repository and serves as a guide to the build. The root file re-exports the namespace, and a dedicated audit module prints the axioms of all headline theorems.

Table 13. The theory modules and their content

Module	Content	Gap	Phase
Geometry.Basic	the double-null arena, areal positivity, the mass aspect	—	I
Geometry.Curvature	Kretschmann and Weyl profiles, the strong and isotropic singularity	G3	II–III
Analysis.HyperbolicPDE	the Einstein–scalar system, constraints, well-posedness	G1	II
Analysis.Interval	the Interval type, operation soundness, the certificate principle	G5	I
Existence.DSS	DSS covariance, the critical solution, existence and uniqueness	G1	II
Existence.ImplicitFunction	the Banach IFT route, perturbation persistence	G1, G5	II
Stability.Linearization	perturbation modes, the linearized operator's interface	G2	III
Stability.Spectrum	the unique unstable mode, $\gamma = 1/\lambda_0$, the scaling law	G2	III
Scaling	universality, the echoing formula	G2	III
Bridge.Certificates	the RatIv type, check interpretations, robustness lemmas	G4	I
Bridge.Generated	exact rational certificate data, the check suites	G4	I
Bridge.Discharge	discharge into the spectral, Newton and interval headlines	G4	I

The import graph repeats the logic of the dependence of results: the arena (Geometry.Basic) feeds both the curvature and the field system; the interval layer serves the implicit-function route; DSS connects the field system with linearization; spectrum and scaling close the physical part; the bridge plugs into all routes as the data channel. Every registry gap is tied to concrete modules, and the status of a registry row can always be verified by building the corresponding subtree.

The build reproduces with two commands: fetching the cached mathlib artifacts, and the build itself, which type-checks every proof and emits the object files. The toolchain and the mathlib revision move together — the rule is fixed in the package configuration and duplicated in the continuous-integration template. The axiom audit runs as a separate command over the built tree and prints the axiom set of each of the thirty-three headline theorems — the very list dissected in chapter 9.

Table 14. The certificate consumption map: producer → check → headline theorem

Kind	The Lean path	Headline discharge
spectrum	producer → Generated: spectrumChecks → the spectrum-shape lemma	bridge_unique_unstable_mode
newton	producer → Generated: newtonChecks → the correction band	correction_root_exists, correction_root_in_band
scaling	producer → Generated: scalingChecks → the agreement with $1/\lambda_0$	scaling_agreement
interval	producer → Generated: intervalChecks → the image soundness	correction_band_image_sound

The consumption map is the shortest summary of the channel: four rows, three links in each. For a reader opening the tree for the first time it is also the excursion route: open a certificate in the artifacts folder, find the corresponding suite in the Generated module, finish at the discharge and print its axioms. The whole excursion takes minutes and proceeds without a single appeal to trust — only to the build.

Appendix C. How to Verify This Volume in One Session

The appendix collects the commands of a full verification into one checklist — from building the theory to regenerating the illustrations. Each row is a standalone step; the failure of any of them signals a desynchronisation of the tree and the text. The full cycle on an ordinary machine takes from several minutes to half an hour, most of the time going to the first download of the library cache.

Table 15. The verification checklist

Step	Command	Expected result
theory build	cd lean && lake exe cache get! && lake build	a green build, object files of all modules
axiom audit	cd lean && lake env lean AxiomAudit.lean	33 theorems, the three standard axioms
service tests	cd python && python -m pytest tests/ -v	65 green tests
certificates: check	cd python && python -m choptuik_service.certify check	committed artifacts match the regeneration
certificates: re-emit	cd python && python -m choptuik_service.certify emit	byte-identical JSONs and the Generated module
live bridge	python choptuik_service/server.py, then the client	a round-trip over the seven methods

Step	Command	Expected result
data	python scripts/generate_data.py	the CSVs match the commit byte-for-byte
figures	python scripts/generate_figures.py --theme light --lang en	the PNGs match those used in the volume
animations	python scripts/generate_animations.py	the GIFs regenerate from seed 1993

The verification of the volume ends by collating two lists: the list of headline discharges of chapter 5 and the axiom prints of step two; they must match element-wise. For the reader who wishes to verify not only the facts but also the illustrations, the theme and language flags of the generators reproduce every picture of the volume — the light theme and Russian or English for the figures, the dark one for the repository's banner art. A successful run of the checklist means that every statement of this volume reproduces on your machine without the author's participation — which was the very goal of the architecture of trust.

Appendix D. The Headline Theorem Ledger and the Milestones

The ledger below lists all thirty-three headline theorems of the axiom audit — twenty-one theoretical and twelve bridge — with a one-line content each. This is the very list printed by the audit module at build time; every row of the table corresponds to an audit line, and a divergence between them is impossible by construction. For completeness, the ledger of the nine milestones with their statuses is placed alongside — the calendar frame of the programme.

Table 16. The thirty-three headline theorems (the axiom audit)

Area	Theorem	Content
Geometry	divergence_invariant_under_rescaling	reparametrisation invariance of the strength criterion (G3)
Geometry	critical_singularity_is_strong_and_isotropic	conditional core: the certificates imply a strong and isotropic singularity (G3)
Analysis	add_sound	soundness of interval addition (G5)
Analysis	mul_sound	soundness of interval multiplication (G5)
Analysis	certificate_principle	a verified enclosure is a theorem (G5)
Analysis	local_wellposedness	local well-posedness of the field system, weak form (G1)
Existence	critical_solution_exists	scaffold existence of the critical solution (G1)
Existence	dss_fundamental_domain_determination	determination of the DSS fundamental domain (G1)

Area	Theorem	Content
Existence	critical_solution_unique	uniqueness up to scale and translation (G1)
Existence	dss_reparametrization_invariance	DSS reparametrisation invariance (G1)
Existence	newton_map_contractive	contractivity of the Newton map (G1)
Existence	existence_via_implicit_function	existence via the quantitative implicit function theorem (G1)
Existence	newton_zero_unique	uniqueness of the zero in the ball (G1)
Existence	existence_persists_under_perturbation	persistence of existence under perturbations of the seed (G1)
Existence	derivIsomorphism_iff	the derivative-isomorphism criterion for the IFT route (G1, G5)
Stability	unique_unstable_mode	the unique unstable mode, under the certificate hypothesis (G2)
Stability	echo_family_structure	the structure of the family of echo-shifted solutions (G2)
Stability	mass_scaling_law	the scaling law in log-log form (G2)
Scaling	gamma_close_to_published	the derived exponent within tolerance of the calibrated one (G2)
Scaling	universality	universality in conditional form (G2)
Scaling	echoing_formula	the echoing formula (G2)
Bridge	im_odd_multiple	the imaginary parts of the decaying modes are multiples of $2\pi/\Delta$ (G4, G2)
Bridge	spectrum_shape_of_certificate	the spectrum shape derivable from the certificate (G4, G2)
Bridge	newton_kantorovich_local	the local Newton–Kantorovich scheme with certificate constants (G4, G1)
Bridge	spectrumChecks_generated	the spectral certificate check suite (G4)
Bridge	newtonChecks_generated	the Newton certificate check suite (G4)
Bridge	scalingChecks_generated	the scaling certificate check suite (G4)
Bridge	intervalChecks_generated	the interval certificate check suite (G4)
Bridge	bridge_unique_unstable_mode	discharge: the model's unique unstable mode (G4, G2)
Bridge	correction_root_exists	discharge: existence of a correction root (G4, G1)
Bridge	correction_root_in_band	discharge: the root lies in the verified band (G4, G1)
Bridge	correction_band_image_sound	discharge: soundness of the band image (G4, G5)

Area	Theorem	Content
Bridge	scaling_agreement	discharge: the fit agrees with $1/\lambda_0$ within tolerance (G4, G2)

Table 17. The programme's milestone ledger

Milestone	Month	Content	Status
M1.1	1	the environment operational	done
M1.2	5	interval certificates verified	done
M1.3	6	the bridge live: typed round-trips and the certificate channel	partial: the certificate channel is live, the FFI client stays illustrative
M2.1	10	the field system formalized	open
M2.2	12	the numerical seed packaged as Lean data	open
M2.3	18	existence proved	open; the scaffold and the NK core are ready, the RT certificate awaits
M3.1	23	the spectral core proved	open; proved under the certificate hypothesis
M3.2	25	the scaling law derived	open; the log-log form and the exponent agreement are proved
M3.3	27	universality assembled	open

Collating the two tables — the theorems and the milestones — gives the full picture of the state: which statements already stand on the three axioms, and which calendar events separate the present from the finish. Recall that a milestone changes status only with committed acceptance evidence; the statuses of the table are therefore always recomputable from the tree — which the checklist of Appendix C invites you to do.

Appendix E. The Correspondence of the Volume's Chapters and the Tree

The last summary ties the chapters of the volume to the files of the tree they describe — for the reader moving in parallel through the text and the repository. Each row names the entry points of the corresponding chapter; all of them appear in the folder map (section 2.8), and here they are collected by subject.

Table 18. The volume's chapters and the corresponding tree files

Chapter	Tree files
2. State of the programme	README.md, roadmap/milestones.csv, blueprint/gap_registry.csv, data/
3. Interval arithmetic	lean/Choptuik/Analysis/Interval.lean, python/choptuik_service/interval_checks.py
4. The bridge	bridge/protocol.md, python/choptuik_service/client.py, python/choptuik_service/server.py, bridge/examples/LeanClient.lean
5. The certificate channel	python/choptuik_service/certify.py, bridge/certificates/, lean/Choptuik/Bridge/Generated.lean, lean/Choptuik/Bridge/Certificates.lean, lean/Choptuik/Bridge/Discharge.lean
6. Existence	lean/Choptuik/Existence/DSS.lean, lean/Choptuik/Existence/ImplicitFunction.lean, lean/Choptuik/Analysis/HyperbolicPDE.lean
7. Stability	lean/Choptuik/Stability/Linearization.lean, lean/Choptuik/Stability/Spectrum.lean, lean/Choptuik/Scaling.lean
8. The singularity	lean/Choptuik/Geometry/Basic.lean, lean/Choptuik/Geometry/Curvature.lean
9. Quality	lean/AxiomAudit.lean, ci/lean-ci.yml, blueprint/technical_specification.md, blueprint/axiomatic_base.md
Appendix A	docs/glossary.md
Appendix B	blueprint/module_map.md, lean/Choptuik.lean, lean/lakefile.toml

Reading in parallel with the tree is the best way to absorb the programme: every chapter becomes an order of magnitude more concrete when its statements are visible in the sources and the tests. We deliberately kept code listings out of the volume: code lives in the repository, where it compiles and is verified, while the volume is responsible for the map by which one approaches that code without wandering.

References

[1] Choptuik, M. W. Critical behaviour in gravitational collapse of a scalar field. Physical Review Letters, 1993.

[2] Gundlach, C., Martin-Garcia, J. M. Critical phenomena in gravitational collapse. Living Reviews in Relativity, 2007.

[3] Abrahams, A. M., Evans, C. R. Universality and scaling in gravitational collapse of a massless scalar field. Physical Review Letters, 1993.

[4] Evans, C. R., Coleman, J. S. Critical phenomena and self-similarity in the gravitational collapse of radiation fluid. Physical Review Letters, 1994.

[5] Koike, T., Hara, T., Adachi, S. Critical behavior in gravitational collapse. Physical Review Letters, 1995.

[6] Eardley, D. M., Giddings, S. B. Analytical black-hole formation. Physical Review D, 2002.

[7] Reiterer, M., Trubowitz, E. The Choptuik critical solution. Communications in Mathematical Physics / arXiv preprints, 2019.

[8] de Moura, L., Kong, S., Avigad, J., van Doorn, F., von Raumer, J. The Lean theorem prover (system description). CADE, 2015.

[9] The mathlib Community. The Lean mathematical library. CPP, 2020.

[10] Moore, R. E. Interval Analysis. Prentice-Hall, 1966.

[11] Kantorovich, L. V., Akilov, G. P. Functional Analysis. Pergamon Press, 1982.

[12] Isaev, I. K. choptuik_ac_bc: machine-verified research programme (parent repository). GitHub, 2026.

[13] Isaev, I. K. formal_choptuik: from code to machine-checked proof — Volume I of the series. GitHub, 2026.

[14] Isaev, I. K. Gap analysis and proof-assistant selection — Volume II of the series. GitHub, 2026.